

HTML-CSS

Tartalom

1. HTML.....	4
1.1. Block/inline elemek.....	4
1.1.1. Blokk Szintű Elemek.....	4
1.1.2. Inline / Soron Belüli Elemek.....	4
1.1.3. Példák a Különbségekre.....	5
1.2. Elemek egymásba ágyazása.....	5
1.3. Listák.....	5
1.3.1. Rendezett Listák (Ordered Lists).....	6
1.4.1. Rendezetlen Listák (Unordered Lists).....	6
1.5.1. Listák Ágyazása.....	6
1.4. Linkek megjelenítése.....	7
1.4.1. Alapvető Hiperhivatkozás.....	7
1.4.2. Hivatkozás Új Böngészőablakban.....	7
1.4.3. Hivatkozás Egy Dokumentum Egy Másik Részére.....	7
1.5. Táblázatok.....	7
1.6. Űrlapok.....	8
1.7. Szemantikai elemek a weboldalon.....	9
1.7.1. Mért használunk szemantikai elemeket?.....	9
1.7.2. Felépítések összehasonlítása.....	11
1.8. HTML 5.....	12
1.9. Audió és Média elemek.....	12
1.9.1. Audio elemek.....	13
1.9.2. Videó elemek.....	13
1.9.3. YouTube videók beágyazása.....	13
1.10. SVG.....	14
1.10.1. Viewport és Viewbox.....	15
1.10.2. Alakzatok.....	15
1.10.3. Bootstrap SVG Ikonok.....	17
1.10.4. SVG Animációk.....	17
2. SEO, a weboldal keresőoptimalizálása.....	17
2.1. A keresők működése.....	18
2.2. Kereső motorok SPA vagy SSR alkalmazásokon.....	18

2.2.1. Single-Page Applications (SPA)	18
2.2.2. Server-Side Rendering (SSR).....	19
2.2.3. Összehasonlítás.....	19
2.3. Egy felhasználó keresési folyamata:	19
2.3.1. Search Engine Result Pages (SERPs).....	20
2.4. Példa alap meta tartalom felépítésére:	21
2.5. On Page SEO	21
2.5.1. Title (cím, fejezet)	21
2.5.2. URL	22
2.5.3. Content (tartalom)	22
2.5.4. Heading (Fejezetek).....	23
2.5.5. Images (képek)	23
2.6. Off Page SEO	23
3. OpenGraph	24
3.1. Előnéztek tesztelési lehetőségei	24
3.2. A meta adatok, az open graph protokoll elemei	24
3.3. Open Graph teszt localhoston.....	26
4. CSS	28
4.1. Inline, internal és external	28
4.1.1. Inline styling.....	28
4.1.2. Internal styling.....	28
4.1.3. External styling.....	28
4.2. CSS Rule felépítése	29
4.3. Font-family	29
4.4. Szövegek formázása	29
4.4.1. Betűtípus és Méret.....	29
4.4.2. Szövegszín és Igazítás	30
4.4.3. Sorköz és Betűköz	30
4.4.4. Szöveg Díszítés és Stílus	30
4.4.5. Szövegárnyék	30
4.4.6. Text Transform.....	30
4.5. Szelektorok	30
4.5.1. Elem / tag szelektor:	30
4.5.2. Class	30
4.5.3. ID.....	31
4.6. Box model	31
4.7. Mértékegységek.....	32
4.8. Display property	32
4.9. Színek	32
4.10. Float és Clearfix	33
4.10.1. Float.....	34
4.10.2. Clearfix.....	35

4.11. Pszeudo classok és elemek	35
4.11.1. Pszeudo classok	35
4.11.2. Pszeudo elemek	36
4.12. Elemek pozicionálása	36
4.12.1. Static	37
4.12.2. Absolute.....	37
4.12.3. Relatív.....	37
4.12.4. Fixed	37
4.12.5. Sticky	37
4.13. CSS Animációk, Transzformációk és Színátmenetek	37
4.13.1. Színátmenetek és Árnyékok	37
4.13.2. Árnyékok	38
4.13.3. Box-shadow:.....	39
4.14. CSS Animációk, Transzformációk és Színátmenetek	40
4.14.1. Translate (eltolás)	41
4.14.2. Rotate (forgatás)	42
4.14.3. Scale (átméretezés).....	42
4.14.4. Matrix (többszöri transzformáció lehetősége).....	43
4.15. CSS Animációk, Transzformációk és Színátmenetek	44
4.15.1. Átmenetek (transition).....	44
4.15.2. Animációk	46
4.16. Flexbox	47
4.16.1. Flex tengelyek - Justify content.....	47
4.16.2. Flex tengelyek - Align items	48
4.16.3. Flex Grow, Shrink és Wrap	49
4.17. CSS Grid.....	50
4.17.1. CSS határok és területek.....	52
4.17.2. Grid tartalom pozicionálása.....	56
4.17.3. Grid beágyazása grid-be	58
4.18. Reszponzív weboldalak	59
4.18.1. Képernyő tájolása	60
4.18.2. Hasznos Media Query eszközök	60
4.18.3. Aspect ratio.....	60
4.18.4. Példák a Media Queries használatára:	61
4.19. Weboldalak akadálymentesítése.....	62
4.20. Custom Property - CSS változók.....	62
4.20.1. Hogyan hozunk létre CSS változót?	62
4.21. Összetett szelektorok	64
4.21.1. Univerzális szelektor	64
4.21.2. Descendant Selector	65
4.21.3. Child selector.....	65
4.21.4. Adjacent Sibling Selector	66

4.21.5. General Sibling Selector.....	67
4.22. BEM	67
4.22.1. Összetett példa:.....	68

HTML, CSS

1. HTML

Az HTML, vagyis a HyperText Markup Language, egy szabványos jelölő/leíró nyelv, amelyet weboldalak készítésére használnak. Az HTML a weboldalak alapvető építőköve, amely leírja és strukturálja a tartalmat az interneten. Az "HyperText" rész a hiperhivatkozásokra utal, amelyek lehetővé teszik a dokumentumok közötti kapcsolatok létrehozását, míg a "Markup Language" rész arra utal, hogy a nyelv jelöléseket (tag-eket) használ a tartalom strukturálására és formázására.

Az HTML tag-ek segítségével meghatározhatók a különböző tartalomtípusok, mint például szöveg, képek, hiperhivatkozások, listák, táblázatok és sok más. Ezek a tag-ek utasításokat adnak a webböngészőknek arra vonatkozóan, hogy hogyan jelenítsék meg az adott tartalmat a felhasználók számára.

Ügyeljünk az olvashatóságra, az egymásba ágyazott tagek esetében, a "gyerek" elem mindig kezdődjön beljebb a jobb áttekinthetőség érdekében.

Az HTML önmagában csak a tartalom strukturálására és jelölésére szolgál, a vizuális stílusok és interaktivitás hozzáadásához más technológiákra, mint például a CSS (Cascading Style Sheets) és a JavaScript, is szükség van. Az HTML, a CSS és a JavaScript együtt alkotják a modern webfejlesztés alapjait.

[Dokumentáció - MDN](#)

1.1. Block/inline elemek

A HTML elemek két fő kategóriába sorolhatók: blokk szintű (block-level) és soron belüli (inline) elemek. Mindkettőnek sajátos viselkedése és felhasználási módja van a weboldalakon.

1.1.1. Blokk Szintű Elemek

Blokk szintű elemek általában az oldalon való strukturális felosztásra szolgálnak. Ezek az elemek új sort kezdenek és a rendelkezésre álló teljes szélességet elfoglalják (hacsak másképpen nincs meghatározva CSS segítségével). Ezeket általában nagyobb tartalmi részek, mint bekezdések, listák, vagy szakaszok jelölésére használják.

Példák blokk szintű elemekre:

```
<div> - Egy általános célú konténer elem.  
<p> - Bekezdést jelöl.  
<h1>, <h2>, <h3> ... - Címsorokat jelöl, különböző fontossági szintekkel.  
<ul>, <ol> - Pontozott vagy számozott listákat jelöl.  
<li> - Lista elemeket jelöl.  
<table> - Táblázatot jelöl.
```

1.1.2. Inline / Soron Belüli Elemek

A soron belüli (inline) elemek a szöveg részeként jelennek meg és nem kezdenek új sort. Ezeket általában a szöveg belső formázására használják, mint például kiemelés, hiperhivatkozások, vagy stílusos megkülönböztetés.

Példák soron belüli elemekre:

```
<span> – Egy általános célú inline konténer elem.  
<a> – Hiperhivatkozást jelöl.  
<strong> vagy <b> – Félkövér szöveghez.  
<em> vagy <i> – Dőlt szöveghez.  
<img> – Képek beágyazásához, ami technikailag inline elem, bár vizuálisan blokkyszerű lehet.
```

1.1.3. Példák a Különbségekre

Ha egy `<p>` elemet (blokk szintű) használsz, az új sort kezd az oldalon:

```
<p>Ez egy bekezdés.</p>  
<p>Ez egy másik bekezdés.</p>
```

Míg ha `<span>` elemeket (inline) használsz, a szöveg ugyanabban a sorban folytatódik:

```
<span>Ez egy szöveg</span> <span>és itt folytatódik ugyanabban a sorban.</span>
```

Blokk szintű elemek gyakran használhatók az oldal nagyobb strukturális részeinek meghatározására, míg az inline elemek a szövegen belüli apróbb formázásokra vagy funkciókra ideálisak.

1.2. Elemek egymásba ágyazása

Az elemek egymásba ágyazása egy gyakori gyakorlat a HTML-ben, ami lehetővé teszi a weboldalak komplex struktúrájának felépítését. Itt van egy egyszerű példa, amely bemutatja, hogyan lehet egymásba ágyazni különböző típusú HTML elemeket:

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Példa Oldal</title>  
</head>  
<body>  
  <div> <!-- Külső Div Elem -->  
    <h1>Ez egy Címsor</h1> <!-- Blokk Szintű Elem -->  
    <p>Ez egy <strong>kiemelt</strong> bekezdés.</p> <!-- Inline Elem a  
Bekezdésen Belül -->  
    <ul> <!-- Lista Blokk Szintű Elem -->  
      <li>Első Elem</li> <!-- Lista Elem -->  
      <li>Második Elem</li>  
      <li>Harmadik Elem</li>  
    </ul>  
  </div>  
</body>  
</html>
```

1.3. Listák

A HTML-ben két fő típusú lista létezik: a rendezett (ordered) listák és a rendezetlen (unordered) listák. Mindkettőt széles körben használják a weboldalakon információk strukturált megjelenítésére.

1.3.1. Rendezett Listák (Ordered Lists)

A rendezett listák, amelyeket a `<ol>` (Ordered List) tag jelöl, számozott vagy betűrendes elemeket tartalmaznak. Az alapértelmezett megjelenítés számozott, de stílusozható CSS segítségével. Az egyes elemeket `<li>` (List Item) tag-ek jelölik.

```
<ol>
  <li>Első elem</li>
  <li>Második elem</li>
  <li>Harmadik elem</li>
</ol>
```

Ez a kód a következőképpen jelenik meg a böngészőben:

1. Első elem
2. Második elem
3. Harmadik elem

1.4.1. Rendezetlen Listák (Unordered Lists)

A rendezetlen listák, amelyeket a `<ul>` (Unordered List) tag jelöl, pontozott vagy más jelölőkkel ellátott elemeket tartalmaznak. Az alapértelmezett jelölő pont, de ez is szabályozható CSS segítségével. Az elemeket itt is `<li>` tag-ek jelölik.

```
<ul>
  <li>Első elem</li>
  <li>Második elem</li>
  <li>Harmadik elem</li>
</ul>
```

Ez a kód a következőképpen jelenik meg a böngészőben:

- Első elem
- Második elem
- Harmadik elem

1.5.1. Listák Ágyazása

Listákat egymásba is ágyazhatunk a hierarchikus adatok megjelenítéséhez. Ez azt jelenti, hogy egy lista elemen belül egy újabb lista található.

```
<ul>
  <li>Első elem</li>
  <li>Második elem
    <ol>
      <li>Al-elem A</li>
      <li>Al-elem B</li>
    </ol>
  </li>
  <li>Harmadik elem</li>
</ul>
```

1.4. Linkek megjelenítése

A HTML-ben a hiperhivatkozásokat (linkeket) a <a> elem segítségével hozhatjuk létre. A <a> elemnek van egy href attribútuma, amely meghatározza a hivatkozás célját, azaz azt az URL-t, amelyre a felhasználó átirányításra kerül a linkre kattintva.

1.4.1. Alapvető Hiperhivatkozás

```
<a href="https://www.w3schools.com/html/">Látogasson el az W3.com-ra</a>
```

1.4.2. Hivatkozás Új Böngészőablakban

Ha azt szeretnénk, hogy a hivatkozás új böngészőablakban vagy fülön nyíljon meg, használhatjuk a target="_blank" attribútumot.

```
<a href="https://www.w3schools.com/html/" target="_blank">Látogasson el az  
W3.com-ra</a>
```

1.4.3. Hivatkozás Egy Dokumentum Egy Másik Részére

Hivatkozhatunk egy oldalon belül egy meghatározott elemre, például egy adott szakaszra, az elem id attribútumával együtt.

```
<h2 id="section1">Szakasz 1</h2>
```

Hivatkozást erre a szakaszra:

```
<a href="#section1">Ugrás a Szakasz 1-re</a>
```

1.5. Táblázatok

A HTML-ben a táblázatokat a <table> elemmel hozhatjuk létre, ami lehetővé teszi adatok rendezett, rácsos formában való megjelenítését. A táblázatok szerkezete több különböző elemből állhat, mint például táblázat sorok (<tr>), oszlopok (<td> vagy <th>), fejlécek (<thead>), láblécek (<tfoot>), és tartalmi részek (<tbody>).

```
<table>  
  <caption>  
    Nyilvántartás  
  </caption>  
  <thead>  
    <tr>  
      <th>Vezetéknév</th>  
      <th>Keresztnév</th>  
      <th>Kor</th>  
    </tr>  
  </thead>  
  <tr>  
    <td>István</td>
```

```

        <td>Kovács</td>
        <td rowspan="2">33</td>
    </tr>
    <tr>
        <td>Hanna</td>
        <td>Balla</td>
    </tr>
    <tfoot>
        <tr>
            <td>Most Kft.</td>
        </tr>
    </tfoot>
</table>

```

A `<table>` elem jelöli a táblázat kezdetét és végét.

`<thead>` A táblázat fejléc részére szolgál, amely általában tartalmazza a `<th>` elemeket.

`<tbody>` A táblázat fő tartalmi része, amely a sorokat és adatcellákat tartalmazza.

`<tfoot>` A táblázat lábléc része, hasznos összegzések vagy jegyzetek számára.

A `<tr>` elemek (Table Row) sorokat jelölnek.

A `<th>` elemek (Table Header) a fejléc cellákat jelölik, amelyek általában félkövéren és középre igazítva vannak.

A `<td>` elemek (Table Data) az adatcellákat jelölik.

1.6. Űrlapok

Az űrlapok alapvető eszközök a webes interakciókhoz, lehetővé téve a felhasználók számára, hogy adatokat küldjenek el a szervernek. Egy HTML űrlapot a `<form>` elemmel hozunk létre, amely tartalmazhat különböző típusú input elemeket, mint például szövegmezőket, jelölőnégyzeteket és gombokat.

```

<form action="/submit-url" method="post">
    <label for="name">Név:</label>
    <input type="text" id="name" name="name" /><br /><br />

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" /><br /><br />

    <label for="message">Üzenet:</label>
    <textarea id="message" name="message"></textarea><br /><br />

    <input type="submit" value="Küldés" />
</form>

<!-- Fieldset - Legend -->
<fieldset>
    <legend>Személyes adatok</legend>
    <div>
        <label for="name">Név: </label>
    
```



```
<input id="name" name="name" type="text" placeholder="Add meg a neved"><br>
</div>
</fieldset>
```

A `<form>` elem meghatározza az űrlap kezdetét és végét. Az `action` attribútuma jelzi, hova küldjük az adatokat a beküldés után, míg a `method` attribútum meghatározza a küldés módját (általában `post` vagy `get`).

A `<label>` elemek címkéket biztosítanak az egyes `input` elemekhez.

A `<input type="text">` elem egy egysoros szövegbeviteli mezőt hoz létre.

A `<input type="email">` elem hasonló a szövegmezőhöz, de e-mail címek bevitelére optimalizált.

A `<textarea>` elem többsoros szövegbevitelt tesz lehetővé.

Végül, a `<input type="submit">` elem egy gombot hoz létre az űrlap beküldéséhez.

További Input Típusok:

Az űrlapokban sok más típusú inputot is használhatunk, például:

checkbox - Jelölőnégyzetek, több választás megjelölésére.

radio - Rádiógombok, amelyek közül csak egy lehet kiválasztva egyszerre.

password - Jelszómezők, amelyek a beírt karaktereket rejtve tartják.

file - Fájlfeltöltő mezők, amelyekkel a felhasználók feltölthetnek fájlokat.

date, time, color, stb. - Speciális input típusok különböző adatformátumokhoz.

1.7. Szemantikai elemek a weboldalon

A HTML5 bevezetésével számos szemantikus elem került bevezetésre, amelyek célja, hogy jobban strukturálják a weboldalak tartalmát és jelentőségét. Szemantikus elemek használatával a készítők egyértelműbben kifejezhetik, hogy az oldal mely részei felelnek meg bizonyos tartalmi vagy strukturális szerepeknek. Ez nemcsak a fejlesztők számára teszi könnyebbé a kód értelmezését, hanem a keresőmotorok és a különféle segédeszközök számára is javítja az oldalak elérhetőségét és SEO-barátságát.

1.7.1. Mért használunk szemantikai elemeket?

- Az internetes tartalmat szüntelenül kategorizáló és kereső algoritmusoknak nyújtunk segítséget, hogy könnyebben beazonosítsa az elemeket az oldalon. (a [kereső robotokról](#) részletesebben a SEO tananyagban)
- Nekünk fejlesztőnek is nagy segítség, hogy könnyebben azonosítsuk a legfontosabb elemeket egy weboldalon.
- Ha [Screen-reader](#)-t használnak az oldalon, szintén nagy segítség az alkalmazásnak, hogy egyértelműen azonosítsa a kulcselemeket

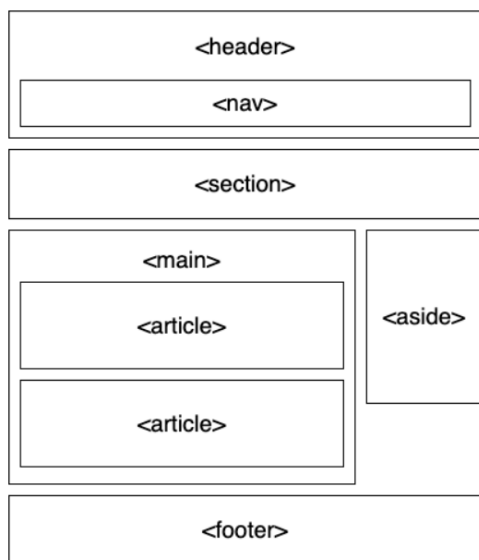
Képzeljük el, hogy 2 fajta kiindulásból házat építünk...

1. Első kiindulási eset, minden alapanyag ömlesztve vegyesen található. Kutakodni kell, hogy a megfelelő anyagot (tégla, cserép, földem stb..) azonosítsuk, majd után kezdjük elhelyezzük a megfelelő helyre
2. Minden fontos építő elemet csoportosítva tárolunk. Így mindig egy helyről használjuk fel az aktuálisan szükséges elemeket

Nem nehéz végig gondolni, melyik esetben lehet gyorsabban és áttekinthetőben felépíteni egy házat (weboldalt :-)

Régebben, gyakran a HTML alap építő "kockáiként" a `<div>`, `<p>`, `<span>` elemeket használtuk csak, ha fel akartuk építeni az oldal vázát, és szerkeszteni a stílus jegyeit.

Ma egy alap szemantikai felépítés így néz ki:



Nézzük meg egy oldal szemantikus elemeit:

- `<header>` ==> Egy oldal, vagy egy nagyobb oldalrész (pl: `<section>` fejléce)
- `<nav>` ==> Linkeket (navigációs sávot) tartalmazó rész
- `<main>` ==> Az oldal beltartalmát összefoglaló rész
- `<section>` ==> Oldal egy alaprésze, amit több bekezdést, képet összefoghat
- `<article>` ==> Egy kiemelt rész, amit több bekezdést, képet összefoghat.
Általában egy cikk tartalmára, oldal többi részétől eltérő tartalmi részre használjuk
- `<aside>` ==> Oldalsó rész (például egy oldalt található kategória list megjelenítése)
- `<details>` ==> Egy tartalom részleteit tartalmazó lenyíló rész
- `<figcaption>` ==> Egy diagram, ábra felirata
- `<figure>` ==> Egy diagram ábra megjelenítése
- `<footer>` ==> Lábléc pozicionálása
- `<mark>` ==> Megjelölt szövegrész (eltérő háttérszín)
- `<summary>` ==> Összegzés
- `<table>` ==> Táblázatot tartalmazó rész
- `<time>` ==> Dátum, idő megjelenítése
- `<em>` ==> *Dőlt betűs* kiemelés (`<i>` elem helyettesítője)
- `<strong>` ==> **Vastag betűs** kiemelés (`<b>` elem helyettesítője)

Kiemelt szerepe van ezeknek az elemeknek, a kereső és felolvasó alkalmazások számára.

Régi példa:

```

<div>
  <p>Ez egy <a href="page.com">link </a> ==> kereső úgy "értelmezi" ez "talán" egy link-et tartalmazó
rész...
</p>
</div>

```

Szemantikus példa:

```

<section>
  <nav>
    <a href=""..> Link 1 ==> kereső robot: "ez egy egyértelmű rész az oldalon, és kell
    <a href=""..> Link 2 benne lennie egy linkek tartalmazó résznek"
  </nav>
</section>

```

Nincs fix szabály, hogyan ágyazhatjuk egymásba a különböző `<section>` és `<article>`, `<aside>` elemeket. Egy cikk-nek lehet több `<section>`-je, egy `<section>` tartalmazhat cikket (`<article>` -t).

1.7.2. Felépítések összehasonlítása

Elsőnek szemantikai elemek nélkül:

```
<div class="top">
  <div class="link-list">
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
  </div>
  <div>
    <div class="container">
      <div>Cim</div>
      <div><i>Alcim</i></div>
      <div>
        <p>
          inventore veritatis et quasi architecto beatae <b>vitae</b> dicta sunt
          explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatu r
          aut odit aut fugit, sed quia consequuntur magni dolores eos qui
          ratione voluptatem sequi nesciunt.
        </p>
        <p>
          Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet,
          orporis suscipit laboriosam, nisi ut aliquid ex ea commodi
          consequatur?
        </p>
      </div>
    </div>
  </div>
  <div>
    <p>Viszlát! Örülök hogy itt jártál</p>
  </div>
</div>
```

Szemantikus elemekkel:

```
<header>
  <nav>
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
  </nav>
</header>
<main>
  <header>
    <h1>Cim</h1>
    <h2><em>Al cim</em></h2>
  </header>
```

```

<section>
  <p>
    inventore veritatis et quasi architecto beatae
    <strong>vitae</strong> dicta sunt explicabo. Nemo enim ipsam voluptatem
    quia voluptas sit aspernatus aut odit aut fugit, sed quia consequuntur
    magni dolores eos qui ratione voluptatem sequi nesciunt.
  </p>
  <p>
    Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, quis
    nostrum exercitationem ullam, nisi ut aliquid ex ea commodi consequatur?
  </p>
</section>
</main>
<footer>
  <p>Viszlát! Örülök hogy itt jártál</p>
</footer>

```

[Dokumentáció – W3](#)

1.8. HTML 5

Az HTML5, amelyet 2014-ben adtak ki hivatalosan, számos új funkciót és elemet hozott be, amelyek jelentősen megkönnyítik a webfejlesztést és javítják a felhasználói élményt. Íme néhány főbb újdonság:

- Új szemantikus elemek: Olyan elemek, mint a <header>, <footer>, <article>, <section>, <nav>, és <aside> segítenek jobban strukturálni az oldalakat és javítják a keresőmotorok számára történő optimalizálást.
- Forma elemek és attribútumok: Az HTML5 új típusú input mezőket vezetett be, például a date, time, email, url, és range, valamint új attribútumokat, mint a placeholder, autofocus, autocomplete, és required.
- Audio és Video támogatás: A <audio> és <video> elemek lehetővé teszik az audio és videó tartalmak beágyazását natív módon, külső pluginok (mint például a Flash) nélkül.
- Canvas és SVG: A <canvas> elem lehetővé teszi a bitmap alapú grafikák dinamikus generálását JavaScript segítségével, míg az SVG a vektoros grafikákhoz nyújt támogatást.
- Web Storage: A Web Storage (lokális tárolás és session tárolás) lehetőséget ad a webalkalmazásoknak adatok tárolására a felhasználó böngészőjében.
- Geolocation API: A Geolocation API lehetővé teszi a weboldalak számára, hogy lekérdezzék a felhasználó földrajzi helyzetét (természetesen a felhasználó engedélyével).
- Offline támogatás: Az offline alkalmazások cache-elésével a HTML5 lehetővé teszi weboldalak számára, hogy offline módban is működjenek.
- Drag and Drop API: A Drag and Drop API egyszerűsíti az elemek egérrel történő húzását és elhelyezését a weboldalakon.
- WebSockets: A WebSockets egy fejlett technológia, ami lehetővé teszi interaktív kommunikációt egy szerver és a felhasználó böngészője között.
- Cross Document Messaging: Biztonságos kommunikációt tesz lehetővé különböző forrású dokumentumok között (pl. iframe-ok esetében).

Ezek a funkciók jelentős előrelépést jelentenek a webes technológiákban, és hatalmas hatással voltak a modern webfejlesztésre.

1.9. Audió és Média elemek

1.9.1. Audio elemek

HTML 5-ös szabvánnyal már megjelent az <audio> elem. Nézzünk egy gyors mini példát:

```
<audio controls>
  <source src="my-music.ogg" type="audio/ogg" />
  <source src="my-music.mp3" type="audio/mpeg" />
  Your browser does not support the audio element.
</audio>
```

A <source> elemekkel lehet több audio típust megadni, mert bizonyos böngészők egy-egy típust jobban kedvelhetnek.

Az első lejátszásra alkalmas elemet indítja el.

Nézzük meg milyen tulajdonságok, legfontosabb attribútumai lehetnek az <audio> elemeknek:

- autoplay: a teljes audio letöltése előtt, az oldal megjelenésekor már elindítja a hangfile-t
- controls: alap vezérlők megjelenítése (play, pause, hangerő)
- loop: folyamatosan visszaugrik az elejére a file végén
- muted: ha használjuk ezt, akkor automatikusa néma lesz az oldal töltődésekor
- src: forrás, elérési út megadása, erre inkább a <source> elem javasolt

példa: <audio loop muted controls>

További audio leírás, tulajdonsághoz kattints [a developer mozilla felületre!](#)

1.9.2. Videó elemek

A <video> elemek nagyon hasonlóan beágyazhatók egy html formátumba:

```
<video width="320" height="240" controls>
  <source src="my-movie.mp4" type="video/mp4">
  <source src="my-movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>
```

A <source> elemekkel lehet több video típust megadni, mert bizonyos böngészők egy-egy típust jobban kedvelhetnek.

Az első lejátszásra alkalmas elemet indítja el.

Nézzük meg most milyen tulajdonságok, legfontosabb attribútumai lehetnek a <video> elemeknek:

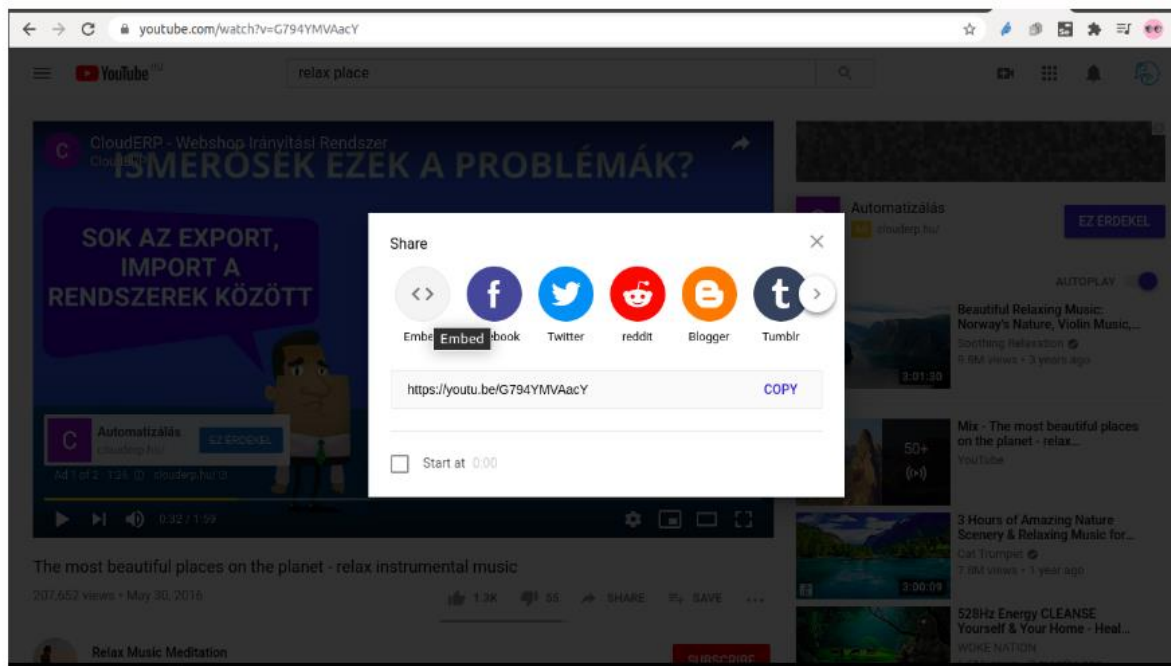
- autoplay: amit elengedő adatot le tudott tölteni, egyből el kezdi a lejátszást
- controls: alap vezérlők megjelenítése (play, pause, hangerő)
- height: videót megjelenítő ablak magassága (abszolút értéket lehet megadni)
- width: videót megjelenítő ablak szélessége (abszolút értéket lehet megadni)
- loop: a videó végén újra lejátszásra kerül az anyag
- muted: hang automatikus némítása
- poster: egy kép megadási lehetősége (url) (mobil telefonok kis képernyőén ajánlott egy induló kép megadása)
- src: videó forrásának megadása (url) mégis inkább a <source> elem használata javasolt

- Mind az audio, mind a video elem esetén elmondható, hogy a legtöbb böngésző által kedvelt formátum az mp3, illetve az mp4.

További <video> leíráshoz, tulajdonságaihoz kattints [a developer mozilla felületre](#)

1.9.3. YouTube videók beágyazása

Van lehetőségünk weboldalunkra, mint "saját" tartalmat beágyazni YouTube videókat. Ehhez nincs másra szükség, mint a kiszemelt YouTube videó alatt a megosztás fülnél az embed típust kiválasztani, majd a felugró ablakból kimásolni az <iframe> tartalmat és elhelyezni az oldalon:



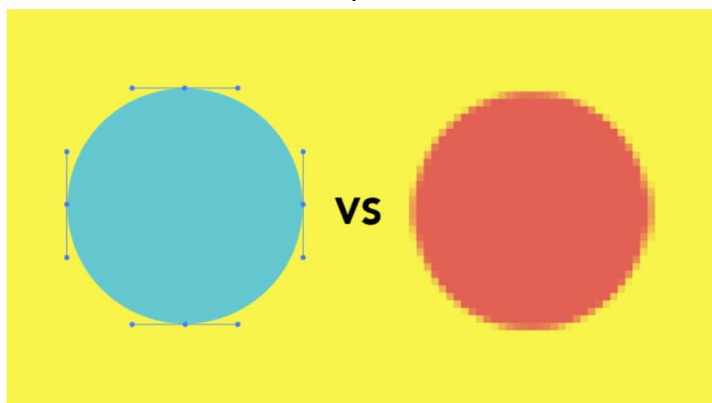
Beilleszthetjük immár, mint saját kis belső "html tartalmat" az oldalunkra

```
<iframe
  width="560"
  height="315"
  src="https://www.youtube.com/embed/ly36kn0ug4k"
  title="YouTube video player"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media;
gyroscope; picture-in-picture"
  allowfullscreen
></iframe>
```

Mint láthatjuk beállíthatjuk a beágyazott videónk szélességi, magassági értékeit is.

1.10. SVG

SVG - Scalable Vector Graphics



Alakzatok, logók és ikonok megjelenítésére szolgáló technika.

Vektorgrafikus formátum: Geometriai alakzatok előállításához használt képábrázolási módszer, melyben az alakzatokat pontok, görbék, sokszögek és egyenesek alkotják, ennek megfelelően koordinátákkal hozzuk létre. Rasztergrafikus formátum: a grafikus elem pixelekből tevődik össze, minden képpontját önállóan definiáljuk. Az SVG legnagyobb előnye, hogy tetszőlegesen nagyítható, nem lesz életlen az alakzat végtelen nagyításban sem. (Innen származik a scalable - skálázhatóság)
A másik előnye, hogy egy nagy felbontású raszteres képnek jelentősen nagyobb a mérete, mint egy vektoros formátumnak, így könnyebben tárolható és kezelhető.

1.10.1. Viewport és Viewbox

Viewport a látható terület mérete, a ViewBox pedig azt kontrollálja, hogy mi látható ezen a területen. Default SVG méret: width=300 és height=150.

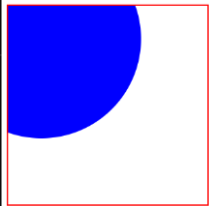
Alapértelmezett ViewBox property:

ViewBox = "0 0 300 150"

Az első koordináta az x, a második az y, a harmadik a szélessége és a negyedik a hosszúsága.

```
<svg  
  viewBox="100 100 300 300"  
  width="300"  
  height="300"  
  style="border: 2px solid red"  
>  
  <circle cx="150" cy="150" r="150" fill="blue" />  
</svg>
```

Ebben a példában nem a 0,0 értékről indul a ViewBox, vagyis a kör egy része nem fog látszódni.



Opcionális olvasnivaló: [SVG Viewport and viewBox \(For Complete Beginners\)](#)

1.10.2. Alakzatok

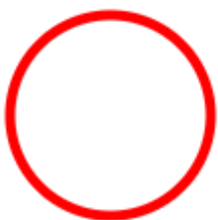
Circle

```
<svg>  
  <circle cx="75" cy="75" r="50" fill="none" stroke="red" stroke-width="5" />  
</svg>
```

cx = Elhelyezzük az x tengelyen

cy = elhelyezzük az y tengelyen

r = a kör sugara



Rectangle

```

<svg>
  <rect
    x="10"
    y="10"
    width="30"
    height="30"
    fill="none"
    stroke="black"
    stroke-width="5"
  />
</svg>
<svg>
  <rect x="10" y="10" rx="10" ry="10" width="100" height="100" fill="red" />
</svg>

```

Az rx és ry attribútumokkal a sarkok kerekítését adjuk meg.



Line

```

<svg>
  <line x1="10" x2="50" y1="110" y2="150" stroke="orange" stroke-width="5" />
</svg>

```



Ellipse

```

<svg>
  <ellipse
    cx="75"
    cy="75"
    rx="20"
    ry="10"
    fill="none"
    stroke="red"
    stroke-width="5"
  />
</svg>

```




Polygon

Szükséges attribútum a points, amellyel megadjuk az x,y koordinátákat.

```
<svg>
  <polygon
    points="200,10 150,120 250,10"
    fill="red"
    stroke="green"
    stroke-width="4"
  />
</svg>
```



1.10.3. Bootstrap SVG Ikonok

<https://icons.getbootstrap.com/>

A Bootstrap SVG ikonokat is biztosít, amelyeket a weboldalakon könnyedén használhatunk.

1.10.4. SVG Animációk

A statikus alakzatok mellett "életre is kelthetjük" a weboldalunkat mozgó SVG elemek segítségével. Ehhez az animate taget kell hozzáadnunk az SVG alakzat taghez.

```
<svg>
  <rect x="50" y="50" width="100" height="100" fill="none" stroke="red">
    <animate
      attributeName="stroke-width"
      values="0;20;0"
      dur="3s"
      repeatCount="indefinite"
    />
  </rect>
</svg>
```

attributeName = az a property, amit animálni szeretnénk

values = kiinduló állapot;megváltozott állapot;végállapot

dur = milyen gyorsan hajtsa végre (mp)

repeatCount = hányszor ismétlje meg - számot is írhatunk értéknek

2. SEO, a weboldal keresőoptimalizálása

A SEO, vagyis a keresőmotor-optimalizálás (Search Engine Optimization) egy digitális marketing stratégia, amelynek célja, hogy növelje egy weboldal láthatóságát a keresőmotorok organikus (nem fizetett) találati

listáiban. A SEO lényege, hogy javítsa az oldal rangsorolását és növelje a forgalmat azáltal, hogy a weboldalt minél relevánsabbá és felhasználóbarátabbá teszi a keresőmotorok számára.

Sokan nem fektetnek nagy hangsúlyt rá, pedig döntő fontosságú, ha egy oldal megjelenését, rangsorolását szeretnék erősíteni az internetes kereső oldalakon. Mik is az általános elemei a SEO-nak?

- taktikák, stratégiai megoldások arra, hogy minél előkelőbb helyen jelenjen meg a weboldal a találatok között
- taktikák, hogy a felhasználók minél több időt töltsenek el az oldalunk, hogy és visszatérjenek hozzánk

Mik a legnépszerűbb keresőmotorok, website-ok?

- Google ==> <https://www.google.com>
- Yahoo ==> <https://yahoo.com>
- Bing ==> <https://bing.com>

A legnépszerűbb közöttük, akit senkinek sem kell bemutatni a Google. Minimális történelmi kitekintés, hogy alakultak ki a mai algoritmusok és rangsorolások a legnépszerűbb keresőmotorokban:

A 2000-2010-es évek között hatalmasat ugrott az internetet látogatók és a weboldalak száma. Sokan meg akarták ragadni a lehetőséget, hogy minél több felhasználót oldalukra csábítsák. Sok jelentéktelen, vagy rossz szándékú oldal is megjelent a piacon.

2011-ben a Google változtatott az algoritmusain ez volt a "Panda Update". Céljai:

- Kizárja a többszöri link felhasználást, olyan oldalakat kontrollálva, amik csak azért jöttek létre, hogy egy adott oldalra mutassanak
- Megtévesztő linkeket tartalmazó oldalak kizárása
- Leginkább releváns találatokat adjon vissza a kereső

Itt található a Google jelenlegi ajánlása, hogyan optimalizáljuk az oldalunkat a Google keresőmotorjához:

[Google SEO guide](#)

2.1. A keresők működése

De hogy is működnek ezek a kereső robotok ([web-crawler](#)-ek)? Két fő feladatuk van:

- Kategorizálása, indexelése az internetes oldalaknak, tartalmaknak
- Keresésre adott eredmények megjelenítése, pontosítása és rangsorolása

Kategorizálás esetén nem a tartalom kategorizálásáról beszélünk. Egy robot nem tudja még pusztán egy szövegből eldönteni, hogy jó, valós adatokat, tényeket közöl vagy sem. Amit felismer az oldal mögött álló HTML tartalom, HTML Tag-ek.

Keresésre adott válaszokban, pontosításokban arra törekszik, hogy az első 2 oldalon a leginkább megfelelő válaszokat adja vissza keresésbe megnevezett feltételnek.

2.2. Kereső motorok SPA vagy SSR alkalmazásokon.

A Single-Page Applications (SPA-k) és a Server-Side Rendering (SSR) eltérő módszereket használnak a weboldalak megjelenítésére és interakciójára, amelyek különböző hatással vannak a keresőmotorok működésére és az SEO-ra.

2.2.1. Single-Page Applications (SPA)

Működési Mód:

- Az SPA-k általában JavaScript keretrendszerekkel, mint például Angular, React vagy Vue.js segítségével készülnek.
- Az SPA csak egy HTML oldalt tölt be, és a JavaScript frissíti a tartalmat a felhasználói interakciók alapján dinamikusan, anélkül, hogy újra betöltené az egész oldalt.

Keresőmotorok és SPA:

- A korai keresőmotorok nehézségekbe ütköztek az SPA-k indexelésénél, mert nem tudtak JavaScriptet futtatni.
- Napjainkban a modern keresőmotorok, mint a Google, jobban kezelik a JavaScript-tartalmakat, de még mindig előfordulhatnak problémák az oldal teljes tartalmának megfelelő időben történő betöltése és indexelése során.

SEO Kihívások:

- Az SPA-k esetében az SEO optimalizálása bonyolultabb lehet, mivel a tartalom dinamikus betöltése nehezítheti a keresőmotorok számára az oldal teljes tartalmának azonosítását.

2.2.2. Server-Side Rendering (SSR)

Működési Mód:

- Az SSR-nél minden egyes oldal vagy oldalrészlet a szerveren generálódik és teljes HTML-ként kerül elküldésre a böngészőbe.
- Minden új oldalbetöltés vagy interakció esetén a szerver új HTML oldalt szolgáltat ki.

Keresőmotorok és SSR:

- Az SSR ideális a keresőmotorok számára, mivel a teljes HTML oldalak könnyen indexelhetők és elemezhetők, mivel a tartalom már a kezdeti betöltéskor rendelkezésre áll.
- Az SSR jobb teljesítményt nyújthat SEO szempontjából, mivel a tartalom azonnal elérhető a keresőmotorok számára.

SEO Előnyök:

- Az SSR oldalak gyorsabban indexelhetők, mivel a tartalom már a kezdeti HTTP válaszban megtalálható.
- A felhasználói élmény is javulhat a gyorsabb betöltési idők miatt, ami pozitívan befolyásolhatja a rangsorolást.

2.2.3. Összehasonlítás

Betöltési Idő: Az SPA-k lassabb kezdeti betöltést kínálhatnak, mivel az egész alkalmazást le kell tölteni először. Az SSR gyorsabb kezdeti betöltést biztosít, mivel a szerver előállítja és elküldi a teljes HTML oldalt.

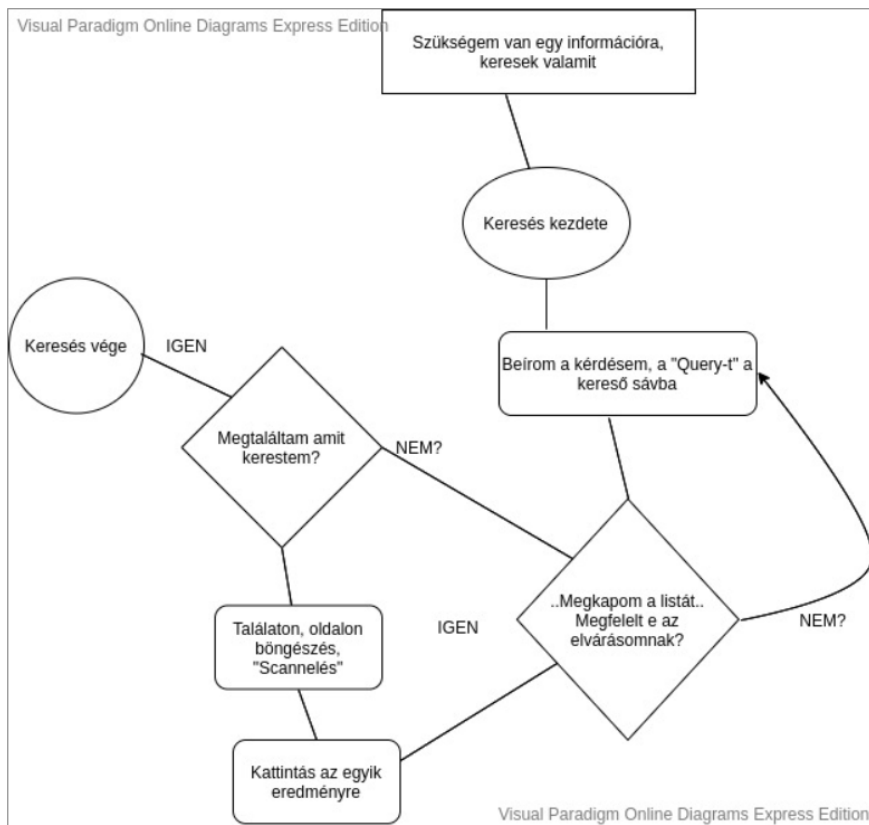
Interaktivitás: Az SPA-k simább felhasználói élményt nyújtanak a dinamikus tartalomfrissítésekkel, míg az SSR-nél minden oldalváltás teljes oldalbetöltést igényel.

SEO: Az SSR általában jobb SEO teljesítményt nyújt, mivel a tartalom azonnal elérhető a keresőmotorok számára. Az SPA-k SEO optimalizálása több erőfeszítést igényel, különösen a JavaScript intenzív tartalmak esetében.

Mindkét technológiának megvannak a maga előnyei és hátrányai, és a választás függ az alkalmazás céljától, a felhasználói interakció típusától, valamint az SEO prioritásaitól. Egyes esetekben a hibrid megoldások, mint például az SSR az első oldalbetöltéshez, majd SPA-szerű interakciók a továbbiakban, optimális megoldást nyújthatnak.

2.3. Egy felhasználó keresési folyamata:

Nézzük meg ehhez az alábbi ábrát:



A fenti ábra jól összefoglalja lényegét, hogy miért fontos nem csak az a szempont, hogy megjelenjen kiemelt helyen az oldalunk, hanem hogy sikerüljön megfogni az a böngésző alanyt, ne lépjen azonnal tovább vagy vissza egy másik weboldalra a találati listában ezzel elhagyva az oldalunkat.

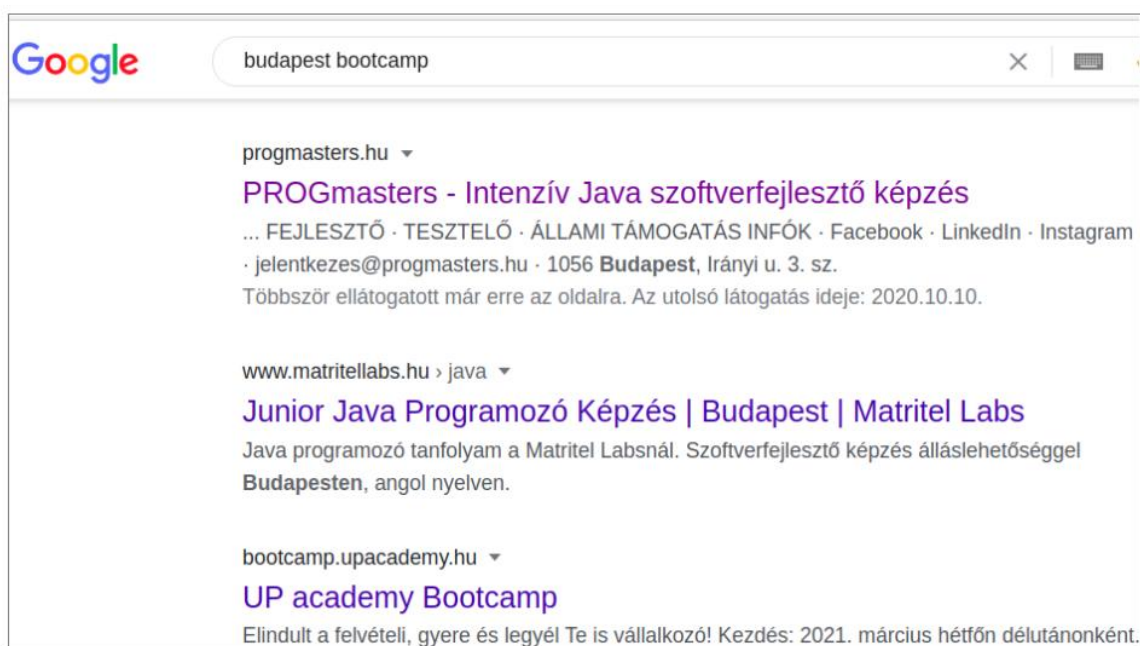
Úgy is kifejezhetjük, ránéz az oldalunkra, a találatra és azonnal kiszűrve a számára fontos dolgokat. Itt jönnek képbe a kulcsszavak (Keyword) fontossága.

- ne csak egy szóba, komplett mondatban fogalmazzuk meg az oldal kulcs mondanivalóját (ne feledjük egy jó weboldal egy választ ad egy kérdésre!)
- kerüljük a túlzott általánosításokat címszavainkban, kulcs mondatainkban. pl: "Ők", "mi ", "Este volt" ..

Azokat a kulcs szavakat, túlzásba vitt "meta adatokat" (így azok az oldalakat is), amik túl sok találatot adnak vissza, kizárják a keresők, hogy gyorsítsák a találatokat.

Nézzük meg, hogy most mit adnak vissza találatok:

2.3.1. Search Engine Result Pages (SERPs)



A találatok két fő elemet jelenítenek meg:

- Title (cím): amit mi fejlesztők adunk meg meta adatként, "meta title"-ként. Maximum 60 karakter lehet.
- Description (leírás): A talált oldal bevezető szövege, amit a kereső motor automatikusan kivág belőle, vagy amit "meta description-ként" adunk meg. (kb. 150-160 karaktert jelenítenek meg a találatban)

2.4. Példa alap meta tartalom felépítésére:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="title" content="Beszédes címkék hasznossága">
  <meta name="robots" content="index,follow">
  <meta name="description" content="Megtanulhatjuk hogy miért fontos a SEO-ban
hogy a jó leírás is">
  <meta name="keywords" content="címké, tanulás, leírás, SEO">
  <title>Document</title>
</head>
```

Továbbiakban nézzük meg, hogy fejlesztőként milyen elemekre kell figyelnünk, hogy növeljük egy oldal "SEO" értékét.

2.5. On Page SEO

5 fő elem csoport van egy weboldal html tartalmában, amivel mi fejlesztők támogatni tudjuk a weboldalunk készítésekor a SEO-t

- Title (cím)
- URL
- Content (Tartalom)
- Headings (Fejlécek)
- Images (Képek)

Ezek megfelelő használata a keresőmotorokat tudja támogatni a kategorizálásban. Ha nem tudja egyértelműen azonosítani az elemeket, témaköröket, akkor kereséskor nem fog megjelenni az oldal.

2.5.1. Title (cím, fejezet)

Itt a HTML felépítés Title címkéjének tartalmáról beszélünk elsősorban:

```
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hogyan építsünk piramist Floating gyakorlással</title>
</head>
<body>
```

- Egyértelműnek kell lennie. Nem lehet zavaró, vagy félrevezető üzenete. Megjelenhet a weboldalon is mint tartalom (pl: címe egy cikknek)
- Oldal tartalmára utalnia kell
- Tartalmazza az oldalon található "key word" -öt, kulcsszavat
- A felhasználó azonnal el tudja dönteni miről szól az oldal
- A felhasználó mindig egy kérdésre keres választ ==> "Megoldást" nyújtson a Title egy kérdésre

Amit a kereső motorok még kifejezetten könnyen értelmeznek:

- 20-60 karakter szám között
- 2 és maximum 20 szó között

Néhány rossz példa:

- <title> képek </title>
- <title> programozó sulis </title>
- <title> gyerekek </title>

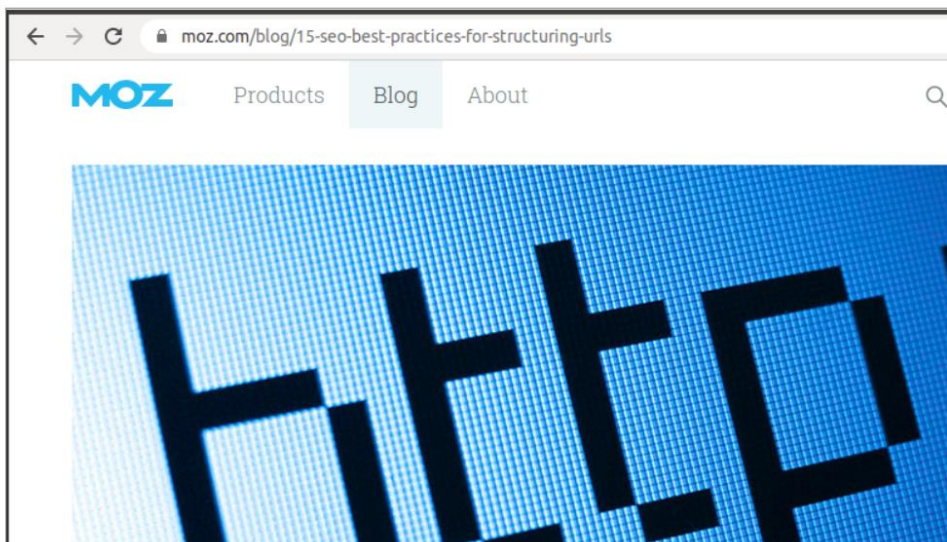
Néhány jó példa:

- <title> ingyenes képek letöltési lehetősége </title>
- <title> legnépszerűbb budapesti programozó iskolák </title>
- <title> hogyan készítsük fel gyermekünk az iskolára </title>

Fontos megjegyzés: egy html tartalom meta adataiban (<meta> címkéinek tartalmaiban) is segíthetünk a kereső motoroknak azonosítani a legfontosabb információkat, de ezeket a legújabb algoritmusok már kezdik figyelmen kívül hagyni, a túlzott meta tartalmak elterjedése miatt.

2.5.2. URL

Egy weboldal és aloldalainak a címsorban megjelenő tartalmairól beszélünk:



- egyértelműen tartalmazza az oldal fő üzenetét
- gyakran megegyezik az oldal Title és URL tartalma (finoman kiegészítője lehet, vagy egy felesleges szóval kevesebb lehet az egyik a másiknak)
- informatív legyen. Lehetőleg ne egy átlagos szó: "cikk", "post", "mi"

Néhány rossz példa:

- myblog/cikk1
- bloggeroldal/post
- familytree-pages/mi

Néhány jó példa:

- myblog/hogyan-készítsünk-tanyasi-levest
- bloggeroldal/legjobb-etterem-tippek-dunakeszin
- familytree-pages/luc-imre-csaladja

2.5.3. Content (tartalom)

Az oldal tartalmi megjelenéséről beszélünk. A kereső mindig összehasonlítja a tartalmat a Title-ben és az URL-ben megfogalmazott tartalommal.

Fontos szempont a felhasználó olvasási élményének növelése, hisz célunk, hogy minél több időt töltsön el oldalunkon!

- Legalább 3 helyen tartalmazza a kulcsszavakat (témát, amit a **Title**-ben megjelöltünk)
- Lehet 1-1 helyen finomítani egy címben szereplő fogalmat: pl eBook ==> digital book
- Egy szekció tartalmi részt bontsunk fel bekezdésekre. 3-4 bekezdés maximum egy szekción belül.
- Max 10-15 sor legyen egy bekezdés
- Bekezdések között tartsunk sőtöréseket
- Használjunk **kiemeléseket** a fontos fogalmakra (dőlt betűk, vastag betűs szavak)
- Ha több gondolatunk, felsorolásunk lenne, használjunk lista elemeket
- Dobjuk fel **képekkel** az oldal tartalmát

2.5.4. Heading (Fejezetek)

Kiemelt fontossága van a megfelelő fejezet címeknek! Itt a **<h1>** -től **<h6>** jelölt html elemekről beszélünk

- Minden oldalnak kell lenni egy, azaz egy darab **<h1>** címkével ellátott tartalomnak, címnek
- A kereső motor egy oldalon a **<h1>** ,**<h2>**, **<h3>** címeket kutatják fel

2.5.5. Images (képek)

Az oldalon megjelenített képekről beszélünk itt, amik nagyban növelik a felhasználó élményt.

- Mindig használjuk a html tartalom **** címkéjében az "alt" attribútumot.
pl: ** saját képek tartalmak készítése, alap lehetőségként ingyenes
- [istockphoto](https://www.istockphoto.com) ==> minimális összegekért lehet képet vásárolni
- [sotryblocks](https://www.sotryblocks.com) ==> minimális összegekért lehet képet vásárolni
- [unplash](https://www.unplash.com) ==> sok ingyenes tartalom

Most már látjuk, hogy kell minél elérhetőbbek tennünk a kereső algoritmusoknak egy oldalt az On Page SEO eszközeivel.

Azonban meg kell említenünk az Off Page SEO lehetőségeket is röviden. Ezek elsősorban a weboldalt megfelelő üzemeltetésével, reklám fogásaival függenek össze.

2.6. Off Page SEO

Mint említettük nem csak meg akarjuk jeleníteni oldalunkat a találatok között. Szeretnénk a felhasználó eltöltött idejét is növelni az oldalunkon. Szeretnénk, hogy miután megtalálták és megfelelően kategorizálták a robotok az oldalunkat, minél kiemeltebb helyen jelenjen meg a találatok között.

Tehát itt a webtartalmak rangsorolásáról beszélünk.

Nézzük mik befolyásolják a találati rangsort:

- Site History: Megfelelő SEO és Szemantikai elemek. Vannak e **spam**-re vagy **hacker** oldalakra utaló linkek?
- Traffic Trends (Forgalom és látogatottság): Nőnek vagy csökkennek az oldal látogatottságai?
- Site Age (az oldal/domain kora): Legalább 1 éves oldal már jó mutatókat hozhat
- Content (Tartalom): Ne legyen az oldalon (felesleges) tartalmi ismétlések
- Az oldalra mutató linkek mennyisége: Hány másik oldalról (domain-ről) van erre az oldalra mutató link, illetve összesen hány darab link mutat az oldalunkra

Összegezhetjük a fentiek alapján, hogy a kereső robotok magát a szöveges tartalmat nem tudják értelmezni. De a látogatottságot és felhasználók bizalmát az oldal felé azt igen!

Könnyű megérteni már, hogy az olyan népszerű oldalak, mint a Facebook, Amazon, Google miért jelennek meg mindig az első találatok között. Nem csak azért, mert "sokan ismerik". Weboldalak ezrei, linkek milliói mutatnak ezekre a webalkalmazásokra.

Így kiemelt szerepe van a SEO-ban a weboldalunkra mutató linkek, mint "**tőké**k" halmozásának. Ezért is fontos, hogy olyan tartalmat készítsünk, amik megoszthatók közösségi felületeken:

- Facebook, Twitter, Instagram
- Fórumokon
- Kommentekben

3. OpenGraph

A közösségi oldalakon történő megjelenés

Első lépésben a talán legismertebb oldal a Facebook fejlesztette ki ezt a protokollt. Céljuk volt hogy a felületükön megosztott különböző internetes tartalmak (posztok) előnézetének megjelenése minimálisan szerkeszthető formában lehessen. Milyen előnyt adhat ez? Ha csak egy embernek szólna a posztunk, szinte mindegy is lehetne, hogy jelenik meg. De az internetes tartalmak zöme, sok embert át akar csábítani a saját weboldalra. Itt már nem mindegy, hogy melyik képet akarjuk megjeleníteni, milyen címmel és milyen bemutatkozó szöveggel stb. Az Open Graph protokoll segítségével egy poszt előnézetének alábbi tartalmi elemei jelölhetők meg:

- Képi, Videó tartalmak megjelenítése
- Címe
- Bemutatkozó szövege
- Tartalom típusa
- Oldaltartalmából részlet
- Cikk író alap adatai, weboldal-linkje
- Link megjelenítése

3.1. Előnéztek tesztelési lehetőségei

A weboldalunk előnézetének megjelenítését, előnézetének megjelenését, több internetes eszköz támogatja.

Nézzünk meg pár fontosabb közösségi oldal Open Graph protokoll leírásait:

- Facebook ==> [open Graph link](#)
- LinkedIn ==> [open Graph link](#)
- Twitter ==> [open Graph link](#)
- Online Open Graph meta tag checker ==> [xyz inline meta tag checker](#)

3.2. A meta adatok, az open graph protokoll elemei

Az említett protokoll elemeit a weboldalunk fejlécében <meta> címkék között jelenítjük meg. Elsőként megadjuk a tulajdonság fajtáját (**property**), majd a hozzátartalmazó értéket (**content**). Itt egy teljesség nélküli példa, ahogy ingatlan portál bemutatkozó oldalának megjelenítése történne:

```
<meta property="og:title" content="Online ingatlan bazár Budapesten" />
<meta
  property="og:description"
  content="Budapest egyetlen igazi online- ingatlan bazára, ahol online is
egymásra talál vevő-eladó"
/>
<meta
  property="og:image"
  content="http://localhost:4200/assets/img/lecsapva2.jpg"
/>
```

Nézzük meg a legalapvetőbb elemeket:

- og:title ==> a bejegyzés címe
- og:type ==> a megosztott tartalom típusa (article, video, music, book, profile website..)
- og:image ==> megosztott kép elérhetősége, url-je
- og:url ==> az oldal url-jének lehetősége
- og:description ==> leírás
- og:video ==> videó tartozik a megosztáshoz, annak elérhetősége
- og:site_name ==> ha a tartalom nagyobb oldal része, akkor a website neve.

Lehet egyes elemeknek még további al-tulajdonságot adni:

```
<meta property="og:type" content="article" />
<meta property="article:publisher" content="https://www.colbyfayock.com" />
```

Itt miután megadtuk a típust, már a cikk-et megjelenítő oldal url-jét is megadhatjuk (vagy más url-t).

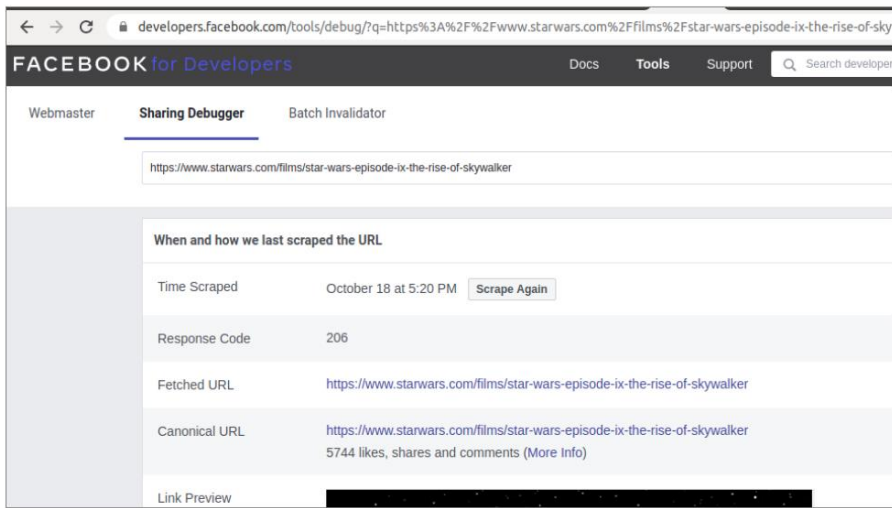
A teljes forráshoz kattints a protokoll webfelületére: [The Open Graph protocol](#)

Nézzünk meg pár tesztelő eszközt saját weboldalunk open graph tag-jeinek vizsgálatára. Fontos információ, hogy ezeken elsősorban az "éles", interneten fellelhető oldalak url-jeket tudjuk vizsgálni.

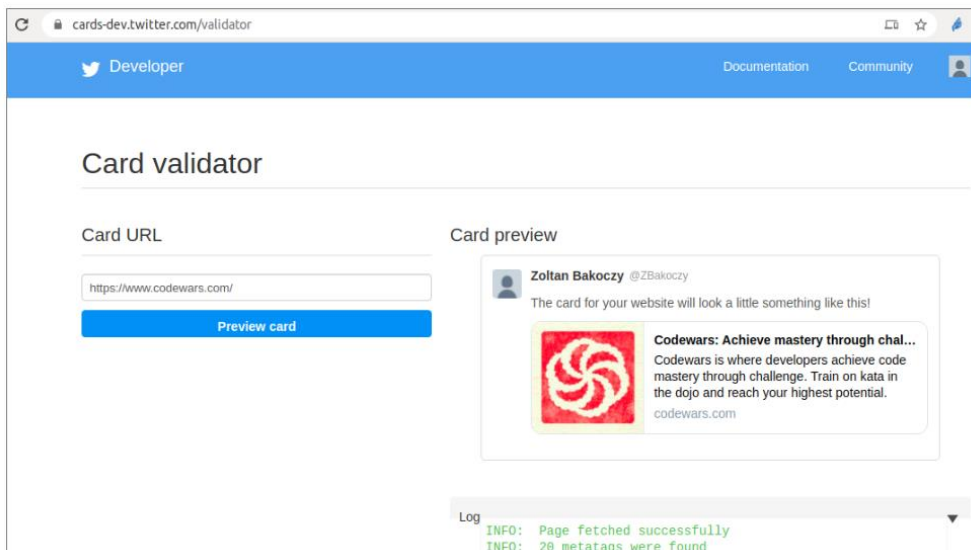
Kattints az alábbi linkekre.

- [Facebook open graph tester](#)
- [Twitter open graph tester](#) (twitter fiók szükséges a használatához)

Nézzük meg elsőnek a Facebook tesztelő felületét és írjuk be egy tetszőleges oldal linkjét.



Láthatjuk ahogy megjelennek a meta adataink a felületen. A HTML-ben lévő <meta> tag-ek alapján felépíti és megjeleníti az előnézetet az oldalon. Így nyomon követhető lehet, hogy a célzott közösségi oldalon milyen változtatások jelent, ahogy átírjuk a különböző meta tartalmakat. Nézzük meg most a [Twitter](#) felületén is ezt a folyamatot. Szintén az URL- címsorába egy tetszőleges oldal, vagy poszt elérhetőségét illesszük be.



Mindkét felületen volt egy közös pont. Egy **URL** elérhetőséget kellett be linkelni, hogy lássuk az előnézetet. Vagyis egy az interneten fellelhető honlapot tudtunk így vizsgálni.

De mi a helyzet ha fejlesztés alatt lévő oldalt akarnák ilyen eszközökkel elérni?

3.3. Open Graph teszt localhoston

Fejlesztés alatt általában nem közvetlen az interneten lévő tartalom szoktunk fejlesztéseket végrehajtani. Védjük az igazi alkalmazást, hogy csak olyan program, kód fusson rajta, amit előtte alaposan leteszteltünk. Ilyenkor egy saját kis belső fejlesztői környezetben, a gépünkön elindult helyi kis szerveren tudjuk futtatni az alkalmazást. Ekkor mondjuk, hogy helyi kiszolgálón, localhost-on fut az alkalmazásunk. Ahhoz, hogy mi is tudjunk ilyen eszközzel élni a VS Code-ban, fel kell telepítenünk a Live Server Extensions-t (bővítményt). Utána nincs más dolgunk, mint jobb egérgomb azon a html-el felületen, amit használni akarunk. Utána kattintsunk az Open With Live Server gombra: És megnyílik az alapértelmezett böngészőnk immár localhoston futtatva az állományunkat. Most keressünk rá az interneten a [Localhost Open Graph Checker](#) felületre. Ez a google bővítményére mutat majd. Adjuk hozzá a Chrome böngészőnkhez a bővítményt. Utána már a böngészőnk fejlécének jobb sarkába az "Extensions" ikonra kattintva elérjük a szolgáltatást (puzzle ikon).

Gyakorló feladat:

Készítsünk egy egyoldalas tetszőleges cikket. Itt most nem számít a szöveges tartalom, és használj minimális stílus formázásokat.

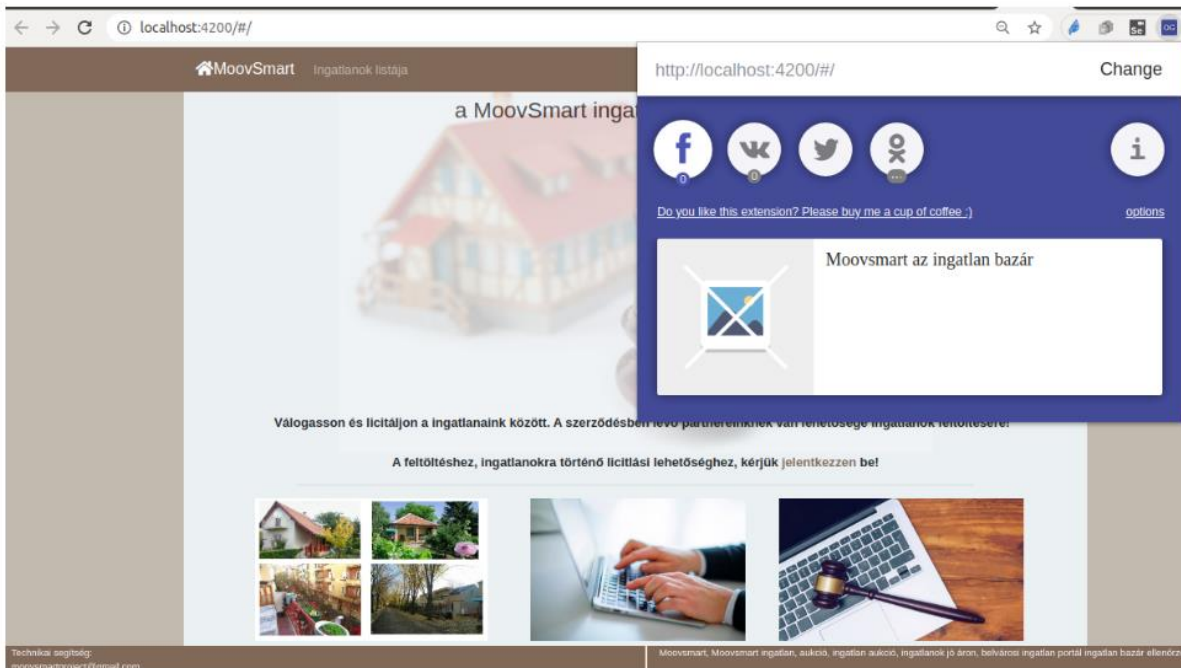
- 1. feladat:

Készítsd el az oldalad

1. legyen egy <h1> cím tag, egy <h2> tag (alcím) <h3> szerző, dátum
2. random szöveg 1 bekezdés (használd nyugodtan egy random latin szöveg generátort)
3. indulásra ne használj semmilyen meta tartalmat.

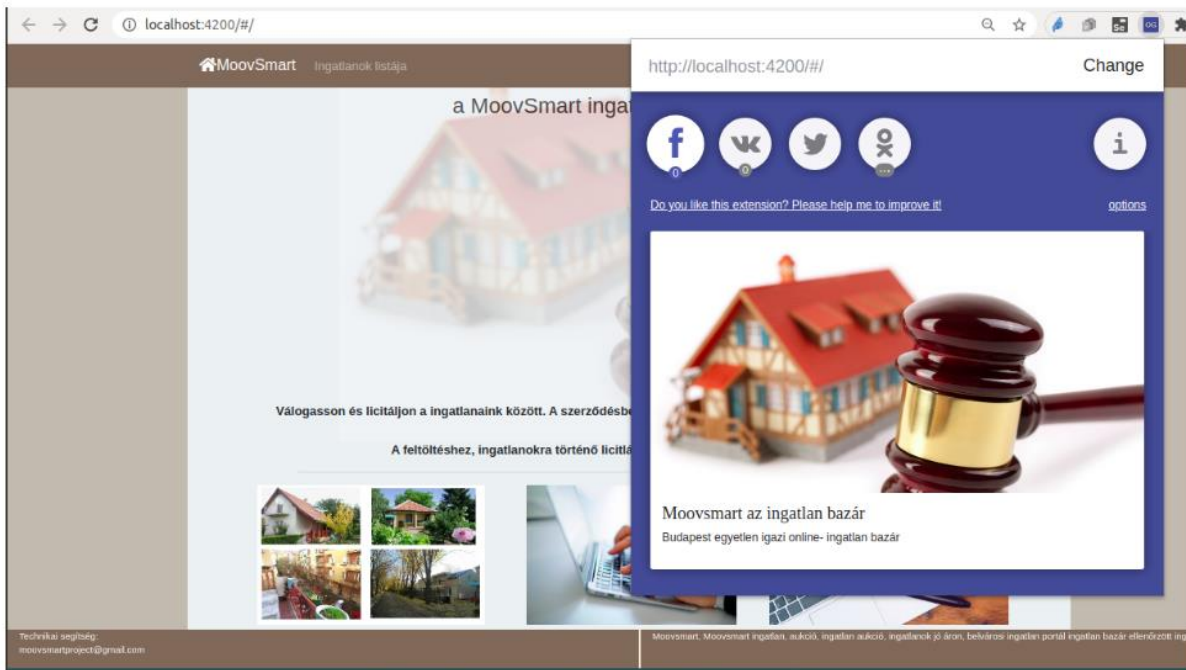
- 2. feladat:

1. indítsd el az oldal futtatását a Live Server-rel
2. Amikor megnyílt az oldalad, kattints az Extension ikonra és válaszd ki az open graph checker-t. (Benne lévő Facebook, Twitter ikonnal is teszteld a nézetet)
3. figyeld meg, milyen előnézetet mutat az oldalad. Legyen itt egy mintakép egy ingatlanos portálról:



- 3. feladat:

1. használj open graph meta adatokat. (**og:title, og:type, og:description, og:image, og:url, article:publisher**)
2. nézd meg a Open Graph Checker Extension-t. figyeld meg a változásokat, alakíts ki egy tetszetős megjelenést
3. válts Facebook és Twitter között is.
4. egészítsd ki pár alap Twitter Open Graph elemmel is (**twitter:card, twitter:title, twitter:description..**)*
5. figyeld meg, hogy változik a megjelenés:



Opcionális olvasmány egy weboldal alap meta adatairól, open graph protocol sémákról ==> [Standard Meta Schemes](#)

4. CSS

A CSS, vagyis a Cascading Style Sheets, egy stílusleíró nyelv, amelyet weboldalak megjelenítésének szabályozására használnak. A CSS a HTML (vagy XML és SVG) elemek megjelenítési stílusát határozza meg a böngészőben. Alapvetően, a CSS-t arra használják, hogy elkülönítsék a tartalom strukturálását és formázását, ami lehetővé teszi a webfejlesztők számára, hogy könnyebben kezeljék és karbantartsák a weboldalakat.

4.1. Inline, internal és external

4.1.1. Inline styling

A formázás közvetlenül a html fájlba, a tagekbe kerül bele. Ez a legkevésbé szerencsés használat, mivel ha több propertyt is beletennénk, akkor hamar átláthatatlan lesz a kód. Az összes opció közül ez a legmagasabb szintű - ha van inline és external css is ugyanarra az elemre, akkor az inline fog érvényesülni - mivel ez van a "legközelebb" az elemhez.

```
<h2 style="background-color: purple">Jó reggelt!</h2>
```

4.1.2. Internal styling

A formázás a HTML fájl <head> elemébe kerül. Ez is kevésbé használt módszer, mivel bár elkülönül a fájlban a formázás, mégis megnöveli a HTML fájl méretét.

```
<head>
  <style>
    h2 {
      background-color: purple;
    }
  </style>
</head>
```

4.1.3. External styling

A formázást egy külön fájlba szervezzük ki. A fájlnek mindig .css kiterjesztésűnek kell lennie, és érdemes beszédes elnevezést adni neki, pl: style.css. A három opció közül ennek a használata javasolt, mivel teljesen elkülöníti a formázást a HTML fájlától. A css fájl is érdemes strukturáltan felépíteni a duplikációk elkerülésének és az olvashatóság elősegítésének az érdekében.

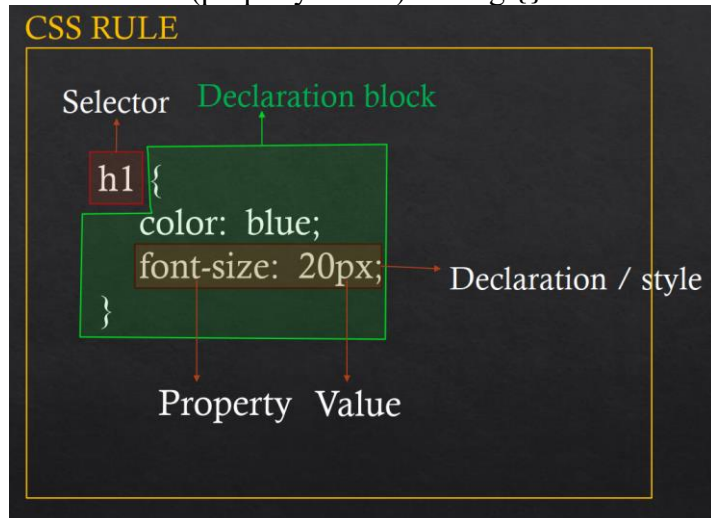
```
<head>
  <link rel="stylesheet" href="style.css" />
</head>
```

4.2. CSS Rule felépítése

A CSS szintaxisa a következő: h2 { background-color: purple; }

- h2 - szelektor
- background-color - property
- purple - value

A deklarációt (property: value) mindig {} közé tesszük, és mindig ;-vel fejezzük be.



4.3. Font-family

A beépített betűtípusok mellett használhatunk külső forrásból is betűtípusokat.

Google: <https://fonts.google.com/>

Ha kiválasztottuk a megfelelő betűtípust, akkor be kell illesztenünk a HTML/CSS fájlba.

HTML (head elem):

```
<link href="https://fonts.googleapis.com/css2?family=Press+Start+2P&display=swap"
rel="stylesheet">
```

CSS (legfelülre érdemes elhelyezni):

```
@import
url(https://fonts.googleapis.com/css2?family=Press+Start+2P&display=swap);
```

4.4. Szövegek formázása

4.4.1. Betűtípus és Méret

```
p {
  font-family: Arial, sans-serif; /* Betűtípus */
  font-size: 16px; /* Betűméret */
}
```

4.4.2. Szövegszín és Igazítás

```
h1 {  
  color: #333333; /* Szövegszín */  
  text-align: center; /* Szöveg középre igazítása */  
}
```

4.4.3. Sorköz és Betűköz

```
body {  
  line-height: 1.5; /* Sorköz */  
  letter-spacing: 1px; /* Betűköz */  
}
```

4.4.4. Szöveg Díszítés és Stílus

```
a {  
  text-decoration: none; /* Eltávolítja az aláhúzást a linkekről */  
  font-style: italic; /* Dőlt betűk */  
}
```

4.4.5. Szövegárnyék

```
h2 {  
  text-shadow: 2px 2px 2px #000000; /* Szövegárnyék */  
}
```

4.4.6. Text Transform

```
.uppercase {  
  text-transform: uppercase; /* Nagybetűssé alakítja a szöveget */  
}
```

4.5. Szelektorok

4.5.1. Elem / tag szelektor:

```
h2 {  
  color: red;  
}
```

4.5.2. Class

Legyen az elnevezése beszédes, kisbetűvel írjuk és kötőjellel választjuk el (ha több szóból áll, de nem kötelező).

HTML:

```
<h2 class="header-text">Szia!</h2>
```

CSS:

```
.header-text {  
  color: red;  
}
```

Miért ezt használjuk?

- Újra felhasználhatóság: Ugyanazt a stílust több elemre is alkalmazhatjuk anélkül, hogy a kódot újra és újra be kellene írunk.
- Karbantarthatóság: A stílusok módosítása egyszerűbb, mivel csak a CSS osztályon belül kell őket frissíteni.
- Rugalmas és Hatékony: A class selectorok segítségével összetett stílusokat hozhatunk létre, amelyeket könnyen alkalmazhatunk különböző elemekre a weboldalon.

4.5.3. ID

HTML:

```
<h2 id="header-text">Szia!</h2>
```

CSS:

```
#header-text {  
  color: red;  
}
```

A class szelektortól az különbözteti meg, hogy egy HTML fájlban egyszerre csak egy helyen használható. Az elnevezési konvenciója ugyanaz, mint a class esetében.

Ha egyszerre több szelektor vonatkozik egy elemre, akkor elem < class < id sorrendben érvényesülnek, vagyis az id-val deklarált formázás fog érvényesülni.

4.6. Box model

A böngésző minden elemre dobozként tekint, innen származik az elnevezés.

Középen maga a tartalom helyezkedik el, erre jönnek rá szintenként:

- padding: belső "padka", amely a tartalmat különíti el a borderen belül.
- border: a szegély, amely határolja az elemet
- margin: az elem körüli tér.

Ezek megadása nem kötelező, ha nem adunk meg nekik értéket, akkor az alapértelmezett értéke 0px lesz.

Mindhárom elem tetszés szerint formázható – top – right – bottom - left sorrendben.

A padding és a border tartozik szorosan az elemhez, a margin már külső komponens. Beállítható, hogy a border és a padding szervesen az elem részét képezze. Érdekes konzekvensnek lenni a szabályok meghatározásakor, ez megkönnyíti a későbbiekben a munkát, így a box-sizing beállítást érdemes a body szelektorba beletenni, így minden benne lévő elem örökölni fogja a beállításokat.

```
body {  
  box-sizing: border-box;  
}
```


Minden a bodyban szereplő elem a borderig "fog nyújtózkodni", vagyis az opcionálisan megadható padding és border méretét is tartalmazni fogja az elem teljes szélessége, vagy magassága.

4.7. Mértékegységek

Ha megadjuk a pontos pixel méretet, akkor abszolút értékadásról beszélhetünk, hiszen minden esetben annyi pixel lesz az elem mérete.

Relatív értékadás:

- **em:** 1em mérete megegyezik a felmenőtől örökölt betűmérettel. Ha ott nincs meghatározva, akkor az alapbeállítás 16px. Ha az örökölnél kisebb méretet szeretnénk adni, akkor kisebb kell, hogy legyen az em értéke 1-nél pl.: 0.8em.
- **rem:** 1rem mérete megegyezik a root - html - elemben meghatározott mérettel, figyelmen kívül hagyja a tartalmazó tag beállításait.

4.8. Display property

A CSS display tulajdonság egy kulcsfontosságú tulajdonság, amely meghatározza, hogy az elem hogyan jelenjen meg a böngészőben. A display tulajdonság szabályozza az elemek elrendezését és viselkedését a dokumentumformázási kontextusban, így meghatározva, hogy az elem blokkszintű, soron belüli, vagy egyéb elrendezési módban legyen.

Leggyakrabban használt értékek:

- **block:** Az elem blokkszintűként viselkedik, ami azt jelenti, hogy új sorban kezdődik és a teljes rendelkezésre álló szélességet elfoglalja. Példák: <div>, <p>, <h1>...
- **inline:** Az elem soron belüliként viselkedik, és csak a szükséges szélességet foglalja el. Nem kezd új sort. Példák: , <a>, ...
- **inline-block:** Kombinálja az inline és block elemek jellemzőit. Olyan, mint az inline elem, de beállítható magassággal és szélességgel.
- **none:** Az elemet elrejt, és úgy viselkedik, mintha az nem is létezne a dokumentumban. Hasznos scripttel vezérelt megjelenítésnél.
- **flex:** Engedélyezi a Flexbox modell használatát, amely modern, rugalmas elrendezési módot kínál.
- **grid:** Engedélyezi a CSS Grid Layout használatát, ami két dimenziós rácsalapú elrendezést tesz lehetővé.

```
h1 {  
  display: none;  
}  
  
h2 {  
  display: inline;  
}
```

4.9. Színek

Alapszínek/név szerinti színek:

Egyszerű, előre meghatározott színeket használhatunk, mint például red, blue, green, stb.


```
p {  
  color: blue; /* Kék szövegszín */  
}
```

Hexadecimális kódok:

6 jegyű (vagy rövidítve 3 jegyű) hexadecimális kódokat használunk a szín meghatározására. Ez a leggyakrabban használt módszer a webfejlesztésben.

```
p {  
  color: #ff5733; /* Egyedi szín hexadecimális kóddal */  
}
```

RGB és RGBA:

Az RGB (Red, Green, Blue) színmodell három alapszín keverésével határoz meg egy színt. Az RGBA egy további alpha értéket is tartalmaz, amely az átlátszóságot szabályozza.

```
p {  
  color: rgb(255, 87, 51); /* RGB szín */  
  background-color: rgba(255, 87, 51, 0.5); /* RGBA szín átlátszósággal */  
}
```

HSL és HSLA:

Az HSL (Hue, Saturation, Lightness) színmodell egy alternatív módszer a színek meghatározására. Az HSLA hasonló az RGBA-hoz, egy további alpha értékkel az átlátszóság szabályozására.

```
p {  
  color: hsl(11, 100%, 61%); /* HSL szín */  
  background-color: hsla(11, 100%, 61%, 0.5); /* HSLA szín átlátszósággal */  
}
```

Számos olyan ingyenes felület van, ahol jól összeválogatott színpaletták közül választhatunk:

<https://coolors.co/palettes/trending>

<https://colorhunt.co/>

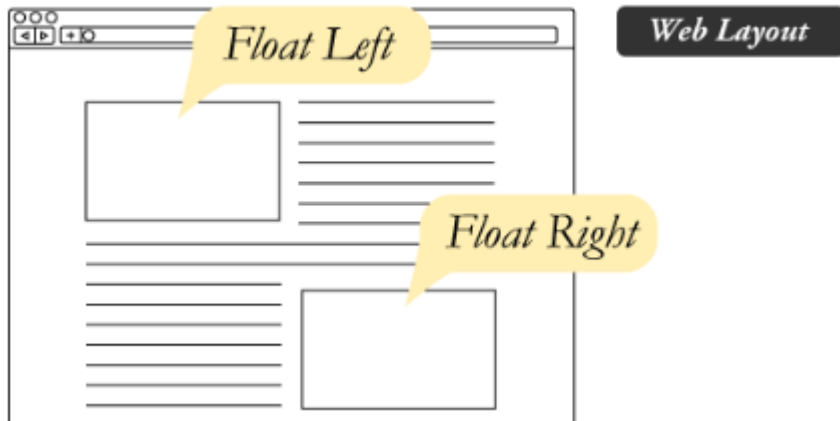
Opcionális olvasnivaló (az anyagát nem tartalmazza sem szintfelmérő, se vizsga):

CSS-Tricks - [Using SVG](#)

CSS-Tricks - [A Guide to SVG Animations](#)

4.10. Float és Clearfix

A CSS float tulajdonsága régebben gyakran használt eszköz volt az elemek elrendezésére, különösen az oldalsávok és a szöveg körbefuttatására. Azonban a modern CSS technikák, mint a Flexbox és a Grid, ma már sokkal hatékonyabb és rugalmasabb megoldásokat kínálnak. Mindazonáltal, a float még mindig hasznos lehet bizonyos esetekben, és fontos megérteni, hogyan működik, valamint hogyan kezelhetők a vele járó problémák, mint például a "clearfix".



Az alapértelmezett Flow:

A html dokumentumot block és inline elemek alkotják. Előbbi egymás alatt, az inline elemek pedig (a szabad hely méretétől függően) egymás mellett helyezkednek el.



4.10.1. Float

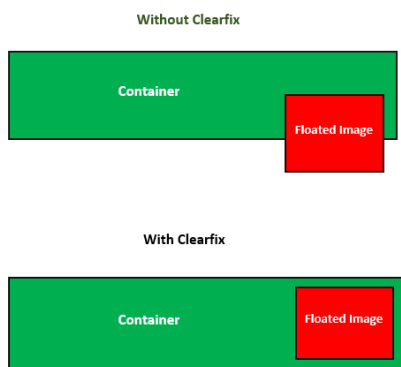
A float tulajdonság lehetővé teszi, hogy az elemeket jobbra vagy balra igazítsuk, így a többi tartalom „körbe folyhatja” őket.

```
<div class="float-left">Ez az elem balra van igazítva.</div>  
<div class="float-right">Ez az elem jobbra van igazítva.</div>
```

```
.float-left {  
  float: left;  
}  
  
.float-right {  
  float: right;  
}
```

4.10.2. Clearfix

A float használatakor problémát jelenthet, hogy a lebegő elemek nem vesznek részt a szülő elem magasságának meghatározásában, ami elrendezési problémákat okozhat. A "clearfix" egy gyakori megoldás erre a problémára. A clearfix technika segítségével biztosíthatjuk, hogy a szülő elem megfelelően magába foglalja a lebegő elemeket is.



```
.clearfix::after {  
  content: '';  
  clear: both;  
  display: block;  
}
```

clearfix::after (szelektor::after) = Pseudo elem, ami az elemhez csatol hozzá tartalmat.
content = ez maga a tartalom, a legtöbb esetben üresen hagyjuk
clear: both = a jobb és a bal float alatt is tiszta lesz a terep
display: block = blokk elemként jeleníti meg

4.11. Pszeudo classok és elemek

4.11.1. Pszeudo classok

A pseudo szó hamisat jelent.

Azért ez a nevük, mert olyan tulajdonságokra vonatkoznak, amelyeket a böngésző automatikusan rendel az elemekhez. Egy lista különböző elemeit tudjuk formázni, de olyan változásokra is tudunk a pseudo osztályokkal formázást írni, melyek külső hatásra következnek be. Ilyen például, ha az elem fölé visszük az egeret vagy kattintottunk már egy linkre.

Az eredeti szelektor után kell kettősponttal írni.

Fontosabb pszeudo osztályok:

- **:hover** (ha az elem fölé viszik az egeret),
- **:visited** (azok a linkek, amelyekre már kattintottak),
- **:active** (ha a felhasználó használ egy elemet, mondjuk rákattint),
- **:focus** (elsősorban input mezőknél, ha éppen használjuk),
- **:first-child** / **:last-child** (a szülő elem első, illetve utolsó gyereke),
- **:nth-child(n)** (az n -edik gyereke a szülő elemnek),
- **:empty** (olyan elem, aminek nincs gyereke, azaz nem tartalmaz beágyazott elemet),
- **:checked** (a kiválasztott checkbox),

- **:disabled** (letiltott elem).

[További pszeudo osztályok itt!](#)

4.11.2. Pszeudo elemek

::before / ::after

A pszeudo elemekkel formázni tudjuk az adott elem egyes részeit, illetve extra tartalmat tudunk hozzáadni a weboldalunkhoz anélkül, hogy a plusz html elemeket kelljen hozzáadni az oldalunkhoz. Akkor hasznos, ha nem szeretnénk kódot ismételni (ez elég gyakori eset).

```
<div class="element">PSEUDO-ELEMENTS</div>
```

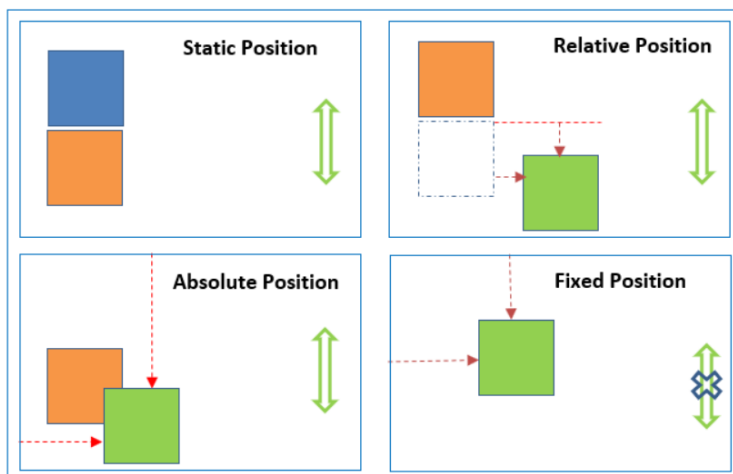
```
.element::before {  
  content: '◀';  
  color: blue;  
}  
  
.element::after {  
  content: '▶';  
  color: red;  
}
```

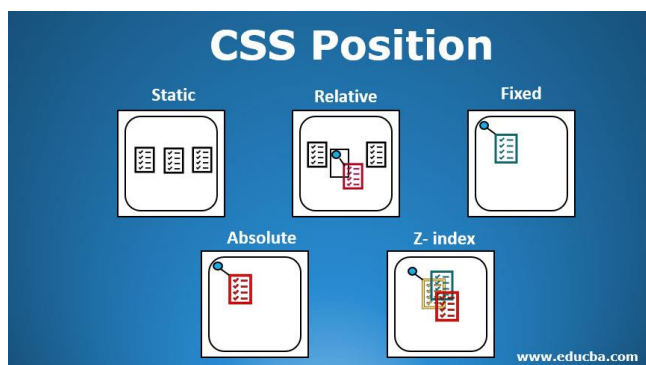
◀ PSEUDO-ELEMENTS ▶

A content attribútum nélkül nem jön létre a pszeudo elem.

Pszeudo elemek esetében nem tudunk összetett szelektorokat használni, külön kell a before és az after elemeket írni a css fájlban.

4.12. Elemek pozicionálása





4.12.1. Static

A statikus position az alapbeállítás, annyit jelent, hogy nincs hozzáadva semmilyen "elmozgató" érték, vagyis úgy jelenik meg az oldalon az elem, ahogy azt a normal flowban elvárjuk. Csak akkor szoktuk használni, ha el akarjuk távolítani az akaratlan formázást az elemről, ekkor úgymond alaphelyezetbe állítjuk.

4.12.2. Absolute

Ha a position property absolute értéket kap, akkor alapértelmezetten nem változik meg az elem helyzete, de kikerül a normal html flow-ból (elrendezéséből). A többi elem ezután nem veszi figyelembe. Az abszolút értéknek adhatunk további propertyket, amelyek befolyásolják az elhelyezkedését. Top-bottom-left-right irányba mozgathatjuk el, így állítható be a pozíciója. Ha ezeket az értékeket megkapja, akkor nincs már rá hatással a parent elem.

Egy abszolút elem mindig megkeresi, hogy a parent elemek közül van-e relatív, vagy fixed pozíciójú. Ha talál, akkor ahhoz fog igazodni. Ha nem talál ilyet, akkor alapértelmezetten a bodyhoz igazodik. Különbözik a fixedtől abban, hogy nem lesz mindig ott a képernyőn, hanem elmozdul, ahogy legörgetünk.

4.12.3. Relatív

A relatív positioning annyit jelent, hogy az elem önmagában relatív. Ha nem adunk hozzá további attribútumokat, akkor nem fogja befolyásolni az elhelyezkedését (olyan, mintha static-ot kapott volna). Ha valamilyen irányba elmozdítjuk, akkor kimozdul a normal flow-ból.

Ha parent elemben használjuk, akkor figyelni kell arra, hogy befolyásolja az absolute positionnel ellátott gyerek elemek pozícióját.

4.12.4. Fixed

A fixed position relatíven igazodik a Viewporthoz, ami nem változik, ha az oldal lejjebb gördül. Ennek köszönhetően a Fixed elem ugyanott marad a képernyőn. Hasznos lehet olyan navbarok esetén, melyeket állandóan láttatni szeretnénk attól függetlenül, hogy a felhasználó az oldal melyik részét nézi épp. Problémát okozhat a használata, ha pl. fixált elem eltakarja valamelyik alsó elemet az olvasó elől.

4.12.5. Sticky

A position:sticky; elem a felhasználó görgetési pozíciója alapján kerül pozicionálásra.

A "ragadós" elem a görgetési pozíciótól függően váltogatja a viselkedését relatív és fixed pozicionálás között. Az elem relatív pozícióban van, amíg egy adott eltolási pozíciót nem ér el a viewportban - ezután "ragad" a helyén (mint a fixed pozícióval rendelkező elemek).

4.13. CSS Animációk, Transzformációk és Színátmenetek

4.13.1. Színátmenetek és Árnyékok

Ahhoz, hogy egy oldal, vagy bármely elem háttérébe színátmenetet varázsoljunk, javasolt elsőnek kitűzni a két vagy több színt, mintegy végpontként. Ehhez jó kiindulási oldal lehet egy rgb color picker, hogy segíts kiválasztani a színárnyalatokat. Érdeemes rákérteni, a Google automatikusan kiad egy ilyen beépített eszközt, de itt egy példaoldal is a sok-sok lehetőség közül:

https://www.rapidtables.com/web/color/RGB_Color.html

Válasszunk ki most két színt hexadecimális értéként (pl: #B9B2FC és #74718B). Ha a megrendelő másképpen nem kívánja, és csak egy hangulatot szeretnénk adni az oldalunknak, ne válasszunk egymáshoz képest túl szélsőséges színt.

Miután megvan a két színünk adjuk őket háttérszínnek egy középre állított divnek az alábbi property segítségével:

- background: linear-gradient(to right, #B9B2FC, #74718B);

Példa forráskód: <https://jsfiddle.net/BZoli/fz19tej8/3/>

Így adjuk meg, hogy egy "lineáris átmenettel" milyen színárnyalatok jelenjenek meg. Az irányát is megadhatjuk, ez esetben jobb irányú átmenettel.

Összegezzük most, hogy a zárójelben milyen főszereplő tulajdonságok lehetnek:

- Ami mindenképpen kell az minimum két színkód, vesszővel elválasztva. (lehet hex, rgb, alap színek)
- Megadhatunk az elején milyen irányba kövesse az átmenetet: (to right, to left, to bottom mint irány)
- Adhatunk százalékos értéket a megadott színeknek, hogy a rendelkezésre álló hely hány százalékát "birtokolják" pl: linear-gradient(red 10%, green 85%, blue 90%);
- 2-nél több színt is felsorolhatunk. Ha nem adunk meg irányt akkor fentről lefelé halad automatikusan az átmenet.

Még egy fontos tudnivalónk van. Nem biztos, hogy minden böngésző támogatja a gradient-eket. Ezért mindenképpen érdemes egy alap háttérszínt is megadni, biztonság szempontjából:

```
#myDiv {
  height: 200px;
  background-color: red; /* ha nem támogatna a böngésző */
  background-image: linear-gradient(
    red,
    orange,
    yellow,
    green,
    blue,
    indigo,
    violet
  );
}
```

További kis angol nyelvű összefoglaló, és sok mini példa [a w3school oldaláról](#) ..

4.13.2. Árnyékok

Árnyékok segítségével enyhe 3 dimenziós hatást tudunk elérni a weboldalunkon. Segítségükkel ki tudjuk kicsit emelni egy-egy elemünket a weboldalból. Egyik legtipikusabb alkalmazása, ha valamilyen "kártya" jellegű rész megjelenését akarjuk erősíteni vele valahogy így:



Hello Africa

Figyeljük meg, hogyan emeli ki az árnyék a környezetéből. Nézzük meg egy pillanatra a szöveget is alaposabban. Ott is megjelenik árnyék?

Ezek után nézzük meg ennek a forrás kódját css osztályunkban:

```
.img-container {  
  box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.5);  
  width: 200px;  
  max-width: 200px;  
  margin: 5% auto;  
  padding: 5px;  
}  
  
img {  
  max-width: 100%;  
}  
  
.img-container p {  
  text-align: center;  
  text-shadow: 1px 1px 3px rgba(0, 0, 30, 0.5);  
}
```

4.13.3. Box-shadow:

Nézzük meg milyen tulajdonságokat tud felvenni:

```
box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.5);
```

- első paraméter: hogy a vízszintes tengelyen hány pixellel toljuk el (példánkban 0)
- második paraméter: függőlegesen mennyi pixelt toljunk el (4px a példánkban)
- harmadik paraméter: opcionális, blur radius (elmosódottság) (8px a példánkban)
- negyedik paraméter: opcionális, spread radius (kiterjedés, árnyék további növelése) (0px a példánkban)
- ötödik paraméter: szín (érdemes lehet rgba-t használni, hogy az áttetszőséggel, minél természetesebb hatást érjünk el)



A szöveg árnyékolása hasonlóan működik, annyi a különbség, hogy nincs spread radius lehetőség benne.

Egy box-shadow property-nek több árnyékot is lehet adni, vesszővel elválasztva:

```
box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.5), 0 2px 8px 4px rgba(0, 0, 50, 0.5);
```



Itt egy forráskód példa játszunk az árnyékokkal szabadon kicsit: <https://jsfiddle.net/BZoli/62zwt4f3/3/>

4.14. CSS Animációk, Transzformációk és Színátmenetek

Nem csak a háttérben lévő színeket tudjuk manipulálni. Képesek vagyunk egy html tartalom megjelenését, alap viselkedését, méretét is megváltoztatni.

Elemek transzformációi:

Pár példa, hogy eddig milyen css property-vel, hogyan változtattuk meg egy-egy elem viselkedését, alap természetét eddig:

- display: inline, inline-block, none..
- floating
- position: absolute, relative, fixed....
- box-sizing

Most olyan eszközt nézünk, amivel képesek vagyunk még érzékenyebben megváltoztatni egy elem viselkedését.

Transactions:

Hogy az átalakításokat használni tudjuk, a **transform** property-t foguk használni.

Egy gyors mini példa:

```
transform: translate(100px, 200px);
```

4 metódust fogunk elsőnek megvizsgálni:

1. translate() egy elem elmozdítása helyéről koordináta rendszernek megfelelően (x, y tengely)
2. rotate() egy elem elforgatása
3. scale() egy elem méretének manipulálása
4. matrix() homogén 2D transzformációs átalakítás, több transzformáció alapján

Kiindulunk 4 sima "div" -el, amit transzformálni fogunk:



4.14.1. Translate (eltolás)

Kiindulás példa property:

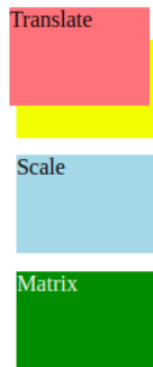
```
transform: translate();
```

Egy vagy két értéket, paramétert vár:

- Egy érték esetén az "X" tengelyen történő pozitív vagy negatív értékben eltolás (pixelben, vagy rendelkezésre álló hely %-ban)
- Két érték esetén mind az x, mind az y tengely mentén eltolja az elemet.

```
.trans {  
  transform: translate(-5px, 60px);  
  background: lightcoral;  
}
```

Eredmény:



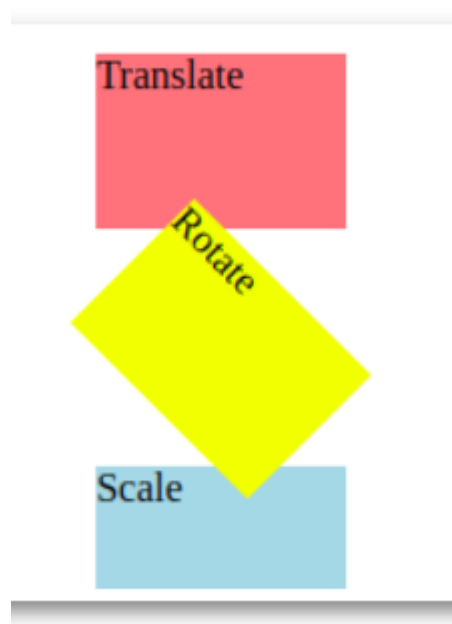
4.14.2. Rotate (forgatás)

Kiindulás példa property:
`transform: rotate();`

Megadhatjuk fokban az elhajlás szögét.

```
.rotate {  
  transform: rotate(45deg);  
  background: yellow;  
}
```

Eredmény:



4.14.3. Scale (átméretezés)

Kiindulás példa property:

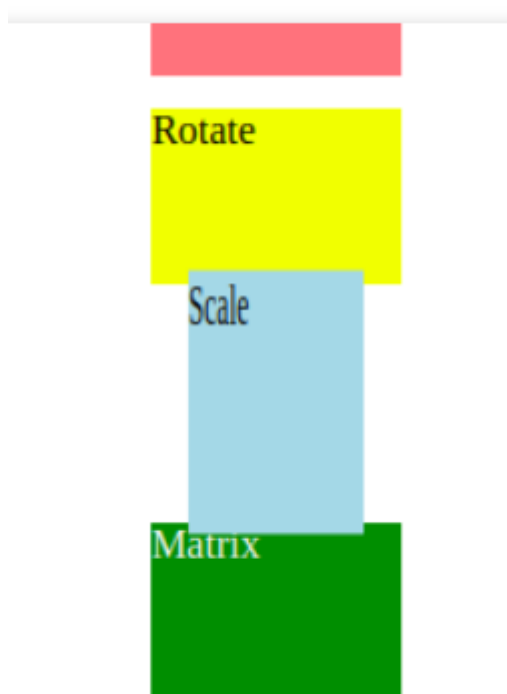
transform: scale();

Egy vagy két értéket, paramétert vár:

- Egy érték esetén mind az x, mind az y tengely mentén nagyítja, kicsinyíti az elemeket (A kiindulási érték, skála: 1)
- Két érték esetén az x, mind az y tengely mentén külön is befolyásolhatom a mértékét

```
.scale {  
  transform: scale(0.7, 1.5);  
  background: lightblue;  
}
```

Eredmény:



Az bemutatott esetben az X tengely mentén 0.7 szerezére csökkentettük a szélességét és 1.5 szerezére növeltük az Y tengely mentén a magasságát.

4.14.4. Matrix (többszöri transzformáció lehetősége)

Kiindulás példa property:

transform: matrix();

Hat paramétert vár, aminek a segítségével egy időben tudjuk megváltoztatni egy elem transzformációit:

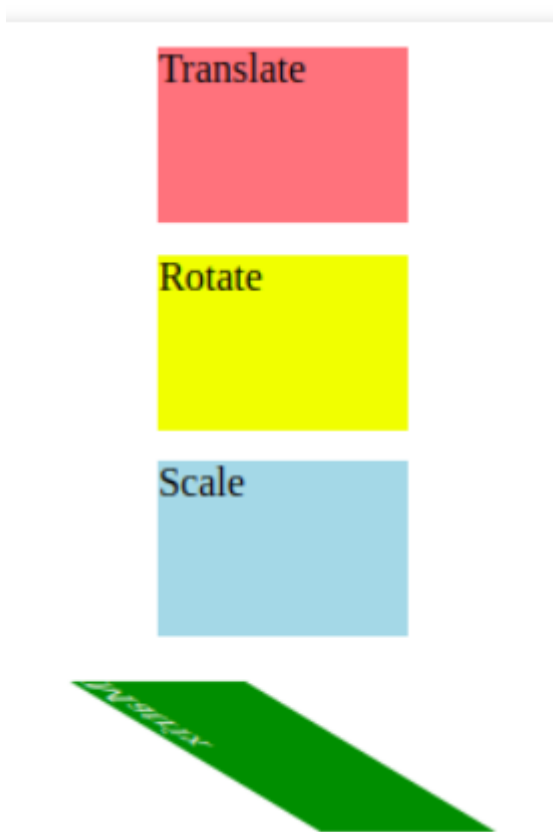
matrix(scaleX(), skewY(), skewX(), scaleY(), translateX(), translateY())

1. X tengely mentén a scaleX(), vagyis a nyújtás
2. Y tengely mentén a [skewY\(\)](#) , vagyis torzítás
3. X tengely mentén [skewX\(\)](#), torzítás
4. Y tengely mentén scaleY(), vagyis nyújtás
5. X tengely történt eltolás translateX()

6. Y tengely mentés történ eltolás translateY()

```
.matrix {  
  transform: matrix(1, 0.6, 1, 0, 0, 0);  
  background: green;  
  color: whitesmoke;  
}
```

Eredmény:



Végül legyen itt egy minta kis forráskód, hogy bátran kísérletezz az elemekkel: <https://jsfiddle.net/BZoli/wh8fazq7/3/>

4.15. CSS Animációk, Transzformációk és Színátmenetek

4.15.1. Átmenetek (transition)

A transition tulajdonság lehetővé teszi, hogy simán változtassunk egy elem stílusát egy állapotról a másikra.

A kulcs property itt a transition.

Nézzünk is egy gyors mini példát:

```
.trans {  
  transform: translate(-5px, 60px);  
  transition: all 3s ease;  
  background: lightcoral;  
}
```

A transition property fő tulajdonságai lehetnek:

- transition-property: melyik tulajdonságokra vonatkozik az átmenet? (esetünkben "all" vagyis minden olyan property-re, ami transzformálható)
- transition-duration: átalakulás időtartama másodpercben (esetünkben 3mp)
- transition-timing-function: milyen legyen az effektus tempója, gyorsuló, vagy lassuló? (példában "ease" könnyed)
- transition-delay: késleltetés másodpercben (opcionális)

Próbáljuk ki az előző oldal [forráskód](#) egy-egy példányán, hogy jelennek meg ezek az effektet.

Lehet nem csak az összes lehetséges transzformációs tulajdonságra("all"), hanem csak egy tulajdonságra is átmenetet készíteni.

Hozunk létre külön egy :hover pszeudo szelektort, ami azt eredményezi, ha egerrel az elem fölé megyünk 300px-re módosítja a szélességét.

Rakjuk be a szélességre vonatkozó következő szabályt az elem eredeti selector body-ba:

transition: width 3s ease;

```
.trans {  
  transform: translate(-5px, 60px);  
  background: lightcoral;  
  transition: width 3s ease;  
}  
  
.trans:hover {  
  width: 300px;  
}
```

Vigyük az egeret a div fölé, és nézzük meg, hogyan változik a viselkedése? Elég egyszerű, és látványos, igaz?

Nézzük meg az effektet (transition-timing-function) milyenek lehetnek:

- ease (lassú kezdet, gyorsul, visszalassul)
- linear (lineáris tempó)
- ease-in (lassuló kezdet)
- ease-out (lassuló vég)
- ease-in-out (lassú kezdet és lassú a vége)

Még egy egyszerű példa:

```
div {  
  transition: background-color 0.5s ease;  
  background-color: blue;  
  width: 100px;  
  height: 100px;  
}  
  
div:hover {  
  background-color: red;  
}
```

Ha fölé visszük az egeret 0.5 másodperc alatt változik a háttérszín kékről pirosra.

4.15.2. Animációk

Az `@keyframes` segítségével bonyolultabb animációkat hozhatunk létre, meghatározva az animáció kulcspontjait.

Nézzünk egy gyors mini példát:

```
.trans {
  position: relative;
  animation-name: go-right;
  animation-duration: 3s;
  animation-iteration-count: infinite;
}

@keyframes go-right {
  from {
    left: 0px;
  }
  to {
    left: 200px;
  }
}
```

`@keyframe` -el és egy névvel (esetünkben "go-right") deklaráljuk, hogy az animációinknak milyen végpontjai lehetnek.

- from a kezdeti állapot (ez lehet 0%)
- to a vég állapot (vagy 100%)

Utána, azon az elementen, amihez kötni akarjuk az animációt, meg kell adni minimálisan a következő 2 tulajdonságot:

- `animation-name: go-right;` (melyik `@keyframe`t használja az elem)
- `animation-duration: 3s;` (késleltetés másodpercben, példánkban 3mp)

További leggyakoribb property-k, tulajdonságok:

- `animation-iteration-count: infinite;` (számlát adhatunk meg, hányszor fusson le, vagy példánkban "infinite", vagyis örökké menjen az animáció)
- `animation-delay: késleltetés mp-ben (pl: 2s)`

Még egy fontos tulajdonságot be kell állítanunk, ha szeretnénk, hogy mozgásos animációja legyen. Az elem `display: property`-jét `relative`-re kell állítanunk, hiszen, kimozdítjuk a "szokásos" környezetéből, ahogy a példánkban is van.

Ekkor már a tudjuk manipulálni példánkban a `left: property`-vel, hogy kimozduljon 200px-el jobbra.

Nézzük meg a forrás kódot és játszunk kicsit vele: <https://jsfiddle.net/BZoli/t193ky5r/16/>

Az animáció végpontjain bármilyen transzformálható tulajdonságát, színét megváltoztathatom, kísérletezzünk

vele bátran:

```
@keyframes rotate {
  50% {
    left: 200px;
    background: green;
  }
  100% {
    left: 0px;
    transform: rotate(360deg);
    background: lightgreen;
  }
}
```

Hozzá a forráskód játéktérnek: <https://jsfiddle.net/BZoli/sh1ou2b0/25/>

4.16. Flexbox



A Flexbox, vagyis a Flexible Box Layout modul, egy modern CSS elrendezési modell, amely lehetővé teszi a konténerek és elemek rugalmas, hatékony elrendezését egydimenziós térben (vízszintesen vagy függőlegesen).

```
selector {
  display: flex;
}
```

Mikor jó nekünk a Flexbox használata? Szinte mindig. Navbar és menük elkészítésénél, rácsos vagy táblázatos megjelenítésnél és például, ha ugyanakkora méretű oszlopokra - hasábokra - van szükségünk.

4.16.1. Flex tengelyek - Justify content

A Flexbox esetében két tengelyben gondolkozhatunk: a vízszintes és függőleges tengelyeknek megvan a maga property-je, amellyel az adott tengely mentén beállíthatjuk az elemeket.

Justify content

Az elemek haladási irány (alapértelmezetten: row) azaz a főtengely szerinti igazításához használjuk.

flex-start



flex-end



center



space-between



space-around



space-evenly



center: középre igazít

flex-start: balra igazít

flex-end: jobbra igazít

space-between: az elemek közötti hely egyenlő lesz

space-around: az elemek körülötti rész marad egyenlő

space-evenly: az elemek minden oldalán ugyanakkora a hely

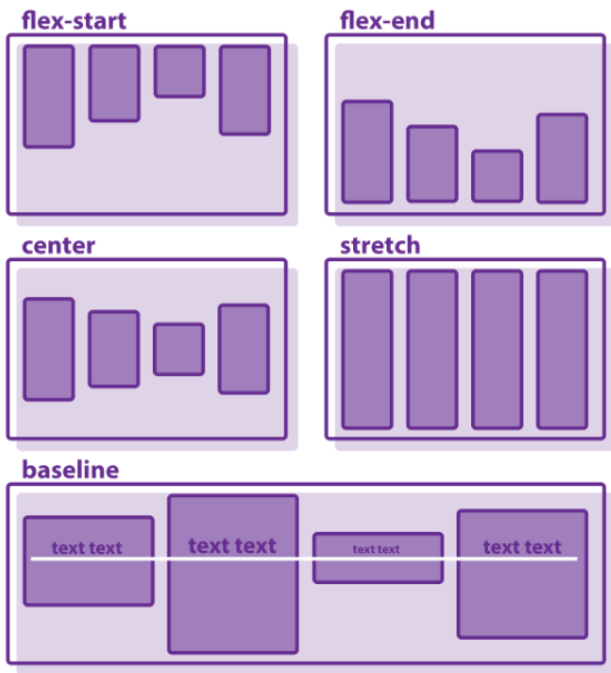
```
selector {  
  justify-content: center;  
}
```

4.16.2. Flex tengelyek - Align items

A Flexbox esetében két tengelyben gondolkozhatunk: a vízszintes és függőleges tengelyeknek megvan a maga property-je, amellyel az adott tengely mentén beállíthatjuk az elemeket.

Align items

Az align-items az elsődleges tengelyre (alapértelmezetten: row) merőlegesen azaz a kereszttengelyen helyezi el az elemeket.



flex-start: Az elemek a konténer tetejére igazodnak.

flex-end: Az elemek a konténer aljára igazodnak.

center: Az elemek a konténeren belül függőlegesen középre igazodnak.

baseline: Az elemek a konténerben a szöveg alapvonalához igazodnak.

stretch: Az elemek széthúzódnak, hogy kitöltsék a konténert.

```
selector {
  align-items: center;
}
```

4.16.3. Flex Grow, Shrink és Wrap

Flex Grow

Azt határozza meg, ha kicsinyítjük az oldalt, akkor a konténer elemei közül melyik maradjon a legnagyobb. Az elemek egymáshoz képest fogják kitölteni a rendelkezésre álló területet. Alapértelmezett érték: 0.

Flex Shrink

Ritkán használjuk, a Flex-grow "ellentéte" - az oldal kicsinyítésekor azt mondja meg, hogy melyik elem tartsa meg legtovább a méretét. Alapértelmezett érték: 1.

Flex Wrap

Ha azt szeretnénk, hogy az oldal kicsinyítésével az elemek egymás alá ugorjanak, akkor használjuk.

```
selector {
  flex-wrap: wrap;
}
```

A nowrap érték az alapértelmezett, illetve azt is beállíthatjuk, hogy az utolsó elem a többi fölé ugorjon az oldal kicsinyítésével. Ehhez a wrap-reverse értéket használjuk.

Gyakorlás:

[Flexbox Froggy](#)

[Flexbox Duck](#) (Flexbox Froggy angolul, kacsával)

[Flexbox defense](#)

Optionális olvasnivaló

[More about Flexbox](#) (animációkkal)

4.17. CSS Grid

A CSS grid az egyik legjobb eszköz arra, hogy egy oldal alap kinézetét, még pontosabban a fő elemek elhelyezkedését rugalmasan megvalósítsuk.

Ezt az alap elrendezést akarjuk a tananyag végére megjeleníteni:



De mindenekelőtt nézzünk meg az alapoktól, hogy épül fel a CSS grid, hogyan tudjuk módosítani az elemeit. Készítsük el indulásra az alábbi alap kódokat:

HTML:

```
<div class="container">
  <div class="one">one</div>
  <div class="two">two</div>
  <div class="three">three</div>
  <div class="four">four</div>
  <div class="five">five</div>
  <div class="six">six</div>
  <div class="seven">seven</div>
</div>
```

CSS:

```
.container {
  color: white;
  font-weight: 600;
  width: 70%;
  margin: 0 auto;
```

```

}

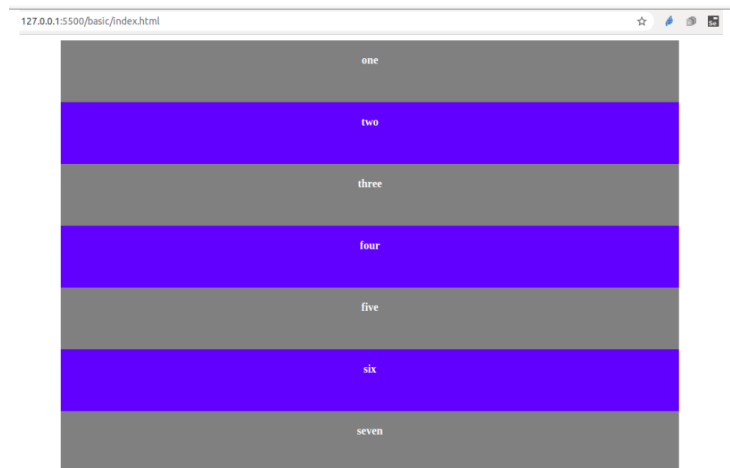
.container div {
  min-height: 70px;
  text-align: center;
  padding-top: 20px;
}

.container div:nth-child(even) {
  background: blue;
}

.container div:nth-child(odd) {
  background: grey;
}

```

Most, ha jól kötöttük mindent össze, akkor ehhez hasonló kinézetet kell kapnunk:



Implementáljunk most bele a kiindulás layout-ba egy grid struktúrát.

display: grid elemmel tudom kijelölni, hol lesz a rácsom. Elsőnek nézzük az oszlopokat:

```

.container {
  color: white;
  font-weight: 600;
  width: 70%;
  margin: 0 auto;
  display: grid;

  /* grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr 1fr; */

  grid-template-columns: 10% 20% 50px 1fr 1fr 1fr 1fr;
}

```

Az oszlopokat megadhatom, px-ben, %-ban, fr, vagyis fraction ("töredék")-ben. Az fr úgy működik, hogy az összes maradék rendelkezésre álló részből mekkora részt vegyen ki. pl:

1fr 2fr 3fr ==> Összesen 3 oszlop, méretnél pedig a területet 1 + 2 + 3 részre bontja fel, vagyis 6 egységnyi szélességre, és ebből 1 egységnyi az első oszlop, 2 egységnyi a második oszlop és így tovább..

Ha sablonos az oszlop méret, és több van belőle használhatjuk az alábbi szintaktikát:

- grid-template-columns: repeat(7, 1fr); ==> hányszor ismételve meg, mekkora egységekben

Van egy nagyon hasznos kis property a grid-gap. Ennek segítségével margint- adhatunk meg a grid elemek köré.

Hozzunk létre oszlopokhoz, hasonlóan sorokat is az alábbi kódrészlet segítségével:

display: grid;

```
/* grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr 1fr; */  
grid-template-columns: repeat(7, 1fr);  
grid-template-rows: 100px 300px 100px;  
grid-gap: 3px;
```

Ha megnézzük a képet nem jelennek meg az új sorok. Biztosan nincsenek ott? De ott vannak csak nincsenek tartalmuk. Játsszunk most kicsit az oszlopok méreteivel:

```
grid-template-columns: repeat(2, 33%);  
grid-template-rows: 100px 300px 100px;  
grid-gap: 3px;
```

Ehhez hasonló képet kellene kapnunk:

one	two
three	four
five	six
seven	

Ha az elemünk nem fér már bele a sorba automatikusan a következő sorba ugrik. Ha nincs elég sor, a grid generál nekünk egyet.

Játsszunk kicsit a kóddal, nézzük meg, hogyan változik a főszereplők elhelyezkedése, ha változtatjuk az oszlopok számát, méreteit stb..

4.17.1. CSS határok és területek

```
grid-template-columns: repeat(3, 33%);
grid-template-rows: 100px 300px 100px;
grid-auto-rows: minmax(100px, auto);
grid-gap: 3px;
```

Ha megnézzük a kódrészletet, van egy új szereplőnk: `grid-auto-rows: minmax(100px, auto);`

Ez úgy működik, ha nem férnek el a szereplők az adott sorban, az automatikusan létrehozott új sorokat 100px magassággal hozza létre, vagy automatikusan kinyújtja, ha nagyobb tartalom kerül bele.

Amúgy alpból létrehoz 3 oszlopot, ami a harmadolja a rendelkezésre álló helyet. Készít 3 sort a megadott méretben és 3px-es távolságot (margin-t) húz a cellák közé.

Ehhez hasonló képet kell kapnunk:



Figyeljük meg a cellák közötti vonalakat, amik most fehérnek tűnnek a háttér miatt. Ezeknek a vonalaknak (cella határoknak) indexei vannak. A következő számozási szabályok vannak:

1. A számozás egytől indul egy sor, vagy oszlopban
2. A terület (a teljes grid szélétől széléig tart)

Vagyis, ha fenti példában a "one" területét nézzük. Akkor mind a sorok, mind az oszlop tekintetében 1-es és a 2 vonal között van.

Akkor kezdjük el átrendezni a felületünket:

```
.one {
  /* grid-column-start: 1; */
  /* grid-column-end: 4; */
  grid-column: 1 / 4;
}
```

A kikommentelt résszel, vagy az alatta lévő összevont egyszerű szintaktikával tudjuk megadni, hogy meddig tartson egy cella.

Manipuláljuk a kettes blokkot is:

```
.two {
  grid-column: 1/2;
```

```
grid-row: 2 / 4;  
}
```

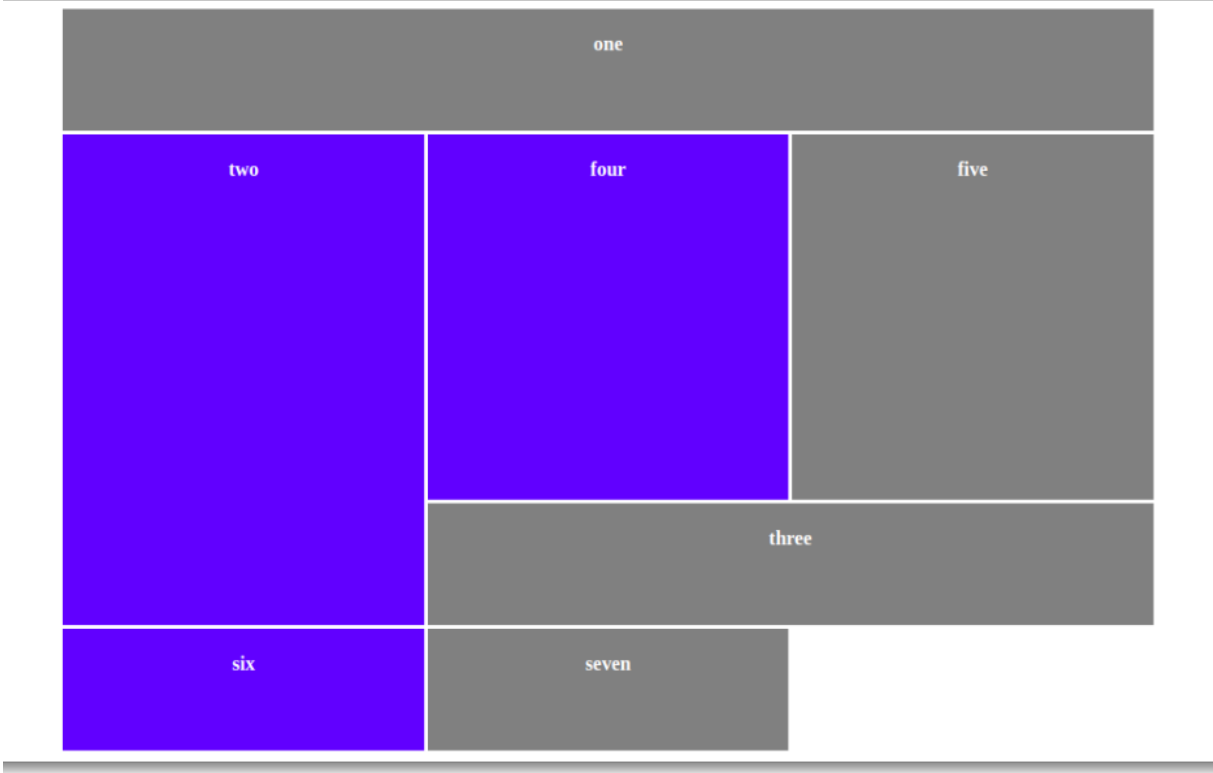
Nézzük meg hogy kezd átrendeződni a felületünk:



Rakjuk be a 3-as blokkot az ötös és hatos helyére:

```
.three {  
  grid-column: 2 / 4;  
  grid-row: 3 / 4;  
}
```

Oszlopok számozása balról jobbra halad, sorok száma fentről lefelé. Nézzük meg az eredményt:



Most próbáljuk meg, önállóan el a 4-es blokkot a 7 mellé rakni, majd az 5-ös blokkot nyújtsuk meg, hogy a 4-es (volt) helyét is felvegye

Ezt az eredményt várjuk:



Hozzá a teljes kódrészlet:

```
.one {
  /* grid-column-start: 1; */
  /* grid-column-end: 4; */
  grid-column: 1 / 4;
}

.two {
  grid-column: 1 / 2;
  grid-row: 2 / 4;
}

.three {
  grid-column: 2 / 4;
  grid-row: 3 / 4;
}

.four {
  grid-column: 3 / 4;
  grid-row: 4 / 5;
}

.five {
```

```
grid-column: 2 / 4;  
}
```

Megadhatunk egy sorban is egy adott területet, négyes elem területe leírható így is:
grid-area: 4 / 3 / 5 / 4;

Itt a sorrend: sor első indexe / oszlop kezdési indexe / sor befejezési indexe / oszlop befejezési indexe.

A következő fejezetben megvizsgáljuk, hogy tudunk egy grid területen belül tovább finomítani az elhelyezéseket.

4.17.2.Grid tartalom pozicionálása

Tartalom rendezése két kulcs property-vel történik a sorokban és oszlopokban:

- align-items: függőleges elrendezés
- justify-items: vízszintes elrendezés

Felvehető legfontosabb paraméterek:

- stretch (default, kitölti a helyet)
- start
- center
- end

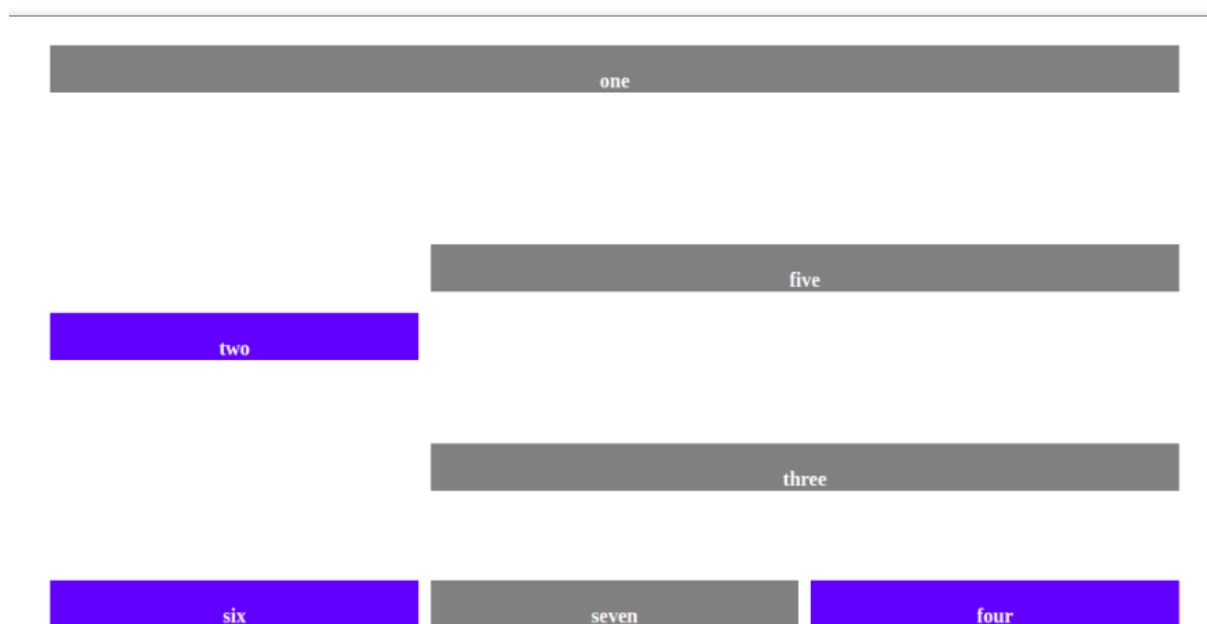
Ahhoz, hogy ennek működését jól láthassuk, vegyük ki a fix méreteiket az elemeknek (ha beállítottuk) Egy példa forráskód:

```
.container {  
  color: white;  
  font-weight: 600;  
  width: 70%;  
  margin: 0 auto;  
  display: grid;  
  grid-template-columns: repeat(3, 33%);  
  grid-template-rows: 100px 200px 100px;  
  grid-auto-rows: minmax(100px, auto);  
  grid-gap: 10px;  
  align-items: center;  
  /* justify-self: end; */  
}  
  
.container div {  
  /* height: 30px; */  
  text-align: center;  
  padding-top: 20px;  
}
```

Játszunk kicsit az elrendezéssel a container-ben. Nézzük meg, hogy változnak a különböző paraméterekre,

mind justify-items (vízszintes irány), mind align-items (függőleges irány) esetén.

A példakód eredményének valami hasonlóan kell kinéznie:



Egy adott grid cella, (area) tartalom elrendezése:

- justify-self: beltartalom elhelyezése vízszintesen
- align-self: beltartalom elhelyezése függőlegesen

Paraméterek megegyeznek a justify-items és az align-items paramétereivel.

Játszunk egy-egy elem saját belső elrendezésével.

Állítsuk vissza a container justify és align-items értékeket ("kommenteljük ki").

Majd egyik elem tulajdonságait változtassuk meg:

```
.five {  
  grid-column: 2 / 4;  
  align-self: center;  
}
```

Hasonló végeredményt kell látnunk:

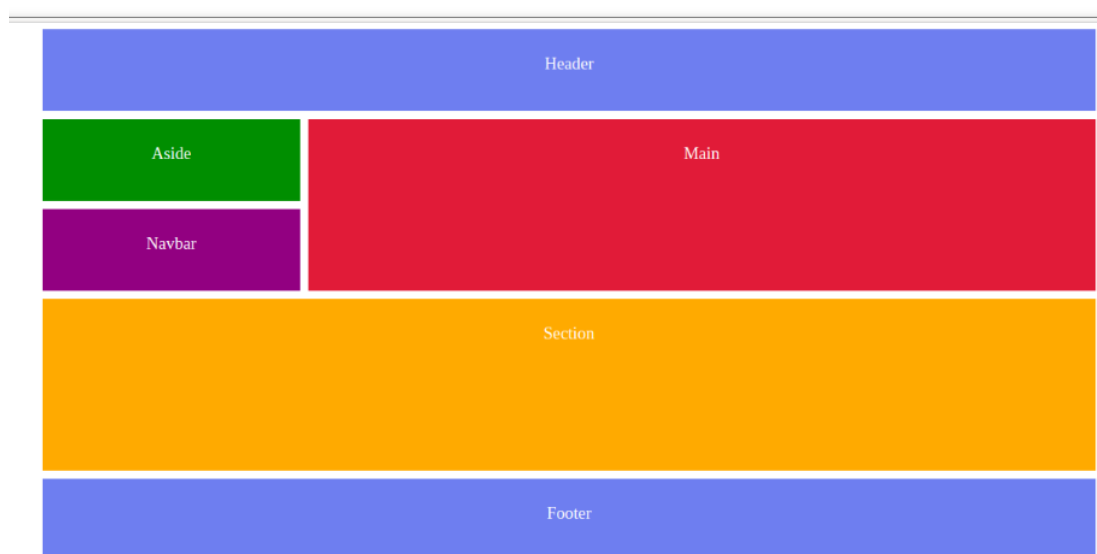


Itt az idő, hogy elkészítsük az induláskor célul kitűzött elrendezést. Ehhez használjunk 12 oszlopos elrendezést. Indulásra egy kódrészlet:

```
.container {
  color: white;
  font-weight: 600;
  width: 70%;
  margin: 0 auto;
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  grid-auto-rows: minmax(100px, auto);
  grid-gap: 3px;
}

.container > * {
  background: grey;
  padding-top: 10px;
  text-align: center;
}
```

Kis gyakorlással már el kell tudnunk készíteni a következő képet:



4.17.3. Grid beágyazása grid-be

Lehet e Grid egyik elemébe beágyazni egy másik grid lay-out-ot? Igen. Bármelyik elembe elhelyezhetünk egy "beágyazott másik Gridet"

Hogy lehet responsive a lay-out?

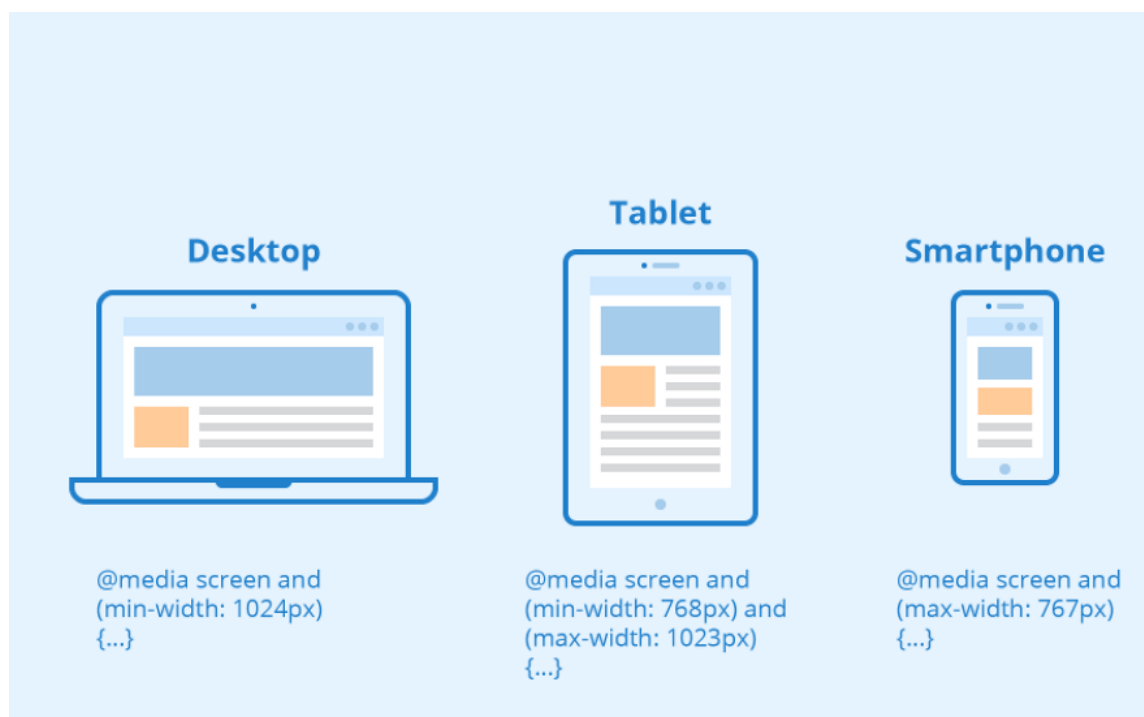
Hát természetesen alap esetben itt is működnek a media query pontok.

Kívánságainknak megfelelően újra rendezhetjük a grid-area- területeit, például, ha mobil nézetben szeretnénk minden elemet teljes szélességre állítani (12 oszlop esetén):

```
@media only screen and (max-width: 600px) {
  aside {
    grid-area: 2 / 1 / 3 / 13;
  }
  nav {
    grid-area: 3 / 1 / 4 / 13;
  }
  main {
    grid-area: 4 / 1 / 5 / 13;
  }
  section {
    grid-area: 5 / 1 / 6 / 13;
  }
  footer {
    grid-column: 1 / 13;
    grid-row: 6 / 7;
  }
}
```

Végezetül a kötelező játék, hogy mélyebben ismerkedjünk a lehetőségekkel a magyar nyelvű [Grid Garden játék](#)
További dokumentumok és leírások: <https://developer.mozilla.org/hu/docs/Glossary/Grid>

4.18. Reszponzív weboldalak



A reszponzív weboldal (RWD) egy olyan megközelítéssel tervezett weboldal, amelynek a célja az, hogy optimális megjelenést biztosítson - könnyű olvashatóság, egyszerű navigáció a lehető legkevesebb átméretezéssel és görgetéssel - a legkülönbözőbb eszközökön (az asztali számítógép monitorjától egészen a mobiltelefonokig). Egy reszponzív elv alapján tervezett oldal tökéletesen igazodik a megjelenítő eszközhöz, mindezt rugalmas felépítéssel, flexibilis képekkel. A CSS3 media query (a @media szabály egy kiterjesztése) segít nekünk definiálni egy ilyen weboldal stíluslapját.

4.18.1. Képernyő tájolása

A képernyő tájolása kissé eltér a készülék tájolásától. Még akkor is, ha egy eszköz nem képes a saját tájolásának érzékelésére, a képernyőn mindig van ilyen opció. A képernyő tájolásának kezelésére többféle mód van, mind CSS-sel, mind pedig JavaScript-el. Az első módszer az orientation media query. Ez lehetővé teszi a tartalom elrendezés CSS használatával történő beállítását, attól függően, hogy a böngészőablak fekvő módban van-e (vagyis szélessége nagyobb, mint a magassága) vagy álló módban (magassága nagyobb, mint a szélessége).

4.18.2. Hasznos Media Query eszközök

Egy átlagos média lekérdezés média típusból (képernyő, nyomtatás, beszéd stb.) és még egy vagy több kifejezésből áll, amelyek olyan médiafunkciókat tartalmaznak, amelyek igaz vagy hamis értéket adnak vissza. A lekérdezés eredménye igaz, ha a média lekérdezésben megadott médiatípus megegyezik annak az eszköznek a típusával, amelyen a dokumentum megjelenik, és a média lekérdezésben szereplő összes kifejezés igaz. A megfelelő stílusokat akkor alkalmazzuk, amikor a média lekérdezés eredménye igaz.

4.18.3. Aspect ratio

Az aspect ratio a viewport magasságát és szélességét jelenti.

Ezek a CSS média lekérdezések a képernyő vagy ablak képarányára (aspect ratio) vonatkoznak. A képarány az ablak szélességének és magasságának aránya. Íme, hogy ezek a lekérdezések hogyan működnek:

Pontos képarány:

```
/* Exact aspect ratio */
@media (aspect-ratio: 2/1) {
  /* Styles */
}
```

Ez akkor alkalmazza a stílusokat, ha a képarány pontosan 2:1, azaz a szélesség kétszerese a magasságnak.

Minimum képarány:

```
/* Minimum aspect ratio */
@media (min-aspect-ratio: 16/9) {
  /* Styles */
}
```

Ez akkor alkalmazza a stílusokat, ha a képarány legalább 16:9, vagyis szélesebb, mint a standard HD képarány.

Maximum képarány:

```
/* Maximum aspect ratio */
@media (max-aspect-ratio: 8/5) {
  /* Styles */
}
```

Ez akkor alkalmazza a stílusokat, ha a képarány legfeljebb 8:5. Ez megakadályozza a stílusok alkalmazását nagyon széles képarányoknál.

Ezek a lekérdezések hasznosak lehetnek olyan esetekben, amikor az oldal stílusát szeretnéd finom hangolni különböző képarányú eszközökön.

Egy rövid példa:

```
img {  
  display: block;  
  margin: auto;  
  width: 100%;  
  height: calc((9 / 19) * 100vw);  
}
```

Itt az `img` elem blokk szintű elemként jelenik meg, középre igazítva. A szélessége 100% a tartalmazó elemének szélességéhez képest. A magassága a képernyő szélességének (`vw` = viewport width) egy bizonyos százalékával van meghatározva, egy 9:19 arány alapján.

```
@media (min-aspect-ratio: 16 / 9) {  
  img {  
    height: 100%;  
    width: calc((19 / 9) * 100vh);  
  }  
}
```

Amikor a képernyő képaránya legalább 16:9, a stílus módosul. Ekkor az `img` elem magassága 100% a tartalmazó elemének magasságához képest. A szélessége a képernyő magasságának (`vh` = viewport height) egy bizonyos százalékával van meghatározva, egy 19:9 arány alapján.

Ez a kód lehetővé teszi, hogy az `img` elem aránya és mérete dinamikusan alkalmazkodjon a képernyő képarányához, biztosítva az optimális megjelenést különböző eszközökön.

4.18.4. Példák a Media Queries használatára:

Alapvető Média Lekérdezés:

```
@media screen and (min-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Ebben a példában a `body` háttérszíne csak akkor vált át világoskékre, ha a képernyő szélessége legalább 600 pixel.

Több Feltétel

```
@media screen and (min-width: 600px) and (max-width: 1200px) {  
  body {  
    background-color: lightgreen;  
  }  
}
```

Itt a body háttérszíne világoszöldre változik, ha a képernyő szélessége 600 és 1200 pixel között van.

Különböző Eszközök

```
@media print {  
  body {  
    font-size: 12pt;  
  }  
}
```

Ebben a példában a body betűmérete 12 pont lesz, amikor a weboldalt nyomtatják.

4.19. Weboldalak akadálymentesítése

Az akadálymentesség az Internetes honlapok kapcsán sem jelent egyébként mást, mint a való életben: elérhetővé, használhatóvá tenni a szolgáltatásokat a sérültek számára is. Ők rendelkeznek olyan felolvasóprogramokkal, speciális eszközökkel, amivel használni tudják az Internetet, azt a közeget, ami néha a legfontosabb kapcsolatot jelenti számukra a külvilággal. A fejlesztők és megrendelők feladata csupán annyi, hogy megfelelő „formába” hozzák honlapjaikat.

Az akadálymentességi munkacsoport által meghatározott (WAI Working Groups – <http://www.w3.org/WAI/>) – első ajánlás a WebContent Accessibility Guidelines 1.0 (WCAG 1.0 – <http://www.w3.org/TR/WCAG10/>) – A második ajánlás a WCAG 2.0 (<http://www.w3.org/TR/WCAG20/>)

További olvasnivaló, gyakorlati példák:

w3schools - [Accessibility Guideline](https://www.w3schools.com/accessibility/)
web.dev/accessibility

Google Lighthouse - [accessibility feature](https://developers.google.com/lighthouse/)

4.20. Custom Property - CSS változók

Értéket adunk egy változónak annak érdekében, hogy a kódban több helyen is felhasználhassuk. A változók használatával jóval áttekinthetőbb lesz a kód, el tudjuk kerülni az ismétlődéseket, és így könnyebben tudunk módosítani az értékeken, mert minden egy helyen van.

Később a Javascript és a preprocessorok is fel tudják majd használni.

4.20.1. Hogyan hozunk létre CSS változót?

Elhelyezése:

```
:root {  
  /* variables */  
}
```

Ez a :root egy pseudo-class, amely a root elemet jelenti a DOM-ban. A HTML-ben a :root a <html> elemmel egyenértékű és megegyezik a html szelektorral, a különbség csak annyi, hogy specifikusabb, mint a html

szelektor. Azzal, hogy a `:root` elemben hozzuk létre a változókat biztosítjuk azt, hogy az oldalon minden elem hozzáférjen a változóhoz.

Lehetőségünk van a `body` elembe is beletenni a változókat, de az áttekinthetőség érdekében használjuk inkább a `:root`-ot. A változókat a CSS fájl tetején hozzuk létre.

Létrehozás:

A változó szintaxisa:

```
:root {  
  --color-primary: #f9ed69;  
  --main-text: 16px;  
  --main-font: 'Roboto', sans-serif;  
  --main-font-color: #000;  
}
```

Használata a CSS szelektorokban

```
.container {  
  background-color: var(--first-color);  
}
```

A property értéke helyett a változót adjuk meg, ezt a `var` kulcsszóval tehetjük meg, majd a változó értékét beletesszük a kulcsszót követő zárójelbe.

```
HTML ▾  
<!DOCTYPE html>  
<html>  
  <head>  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width">  
    <title>JS Bin</title>  
  </head>  
  <body>  
  
    <div>  
      <p>Use CSS variables</p>  
    </div>  
  
  </body>  
</html>
```

CSS

```
CSS ▾  
  
:root {  
  --main-background-color: red;  
  --main-color: white;  
}  
  
p {  
  background-color: var(--main-background-color);  
  color: var(--main-color);  
}
```


 Use CSS variables

Böngészők és a CSS változók

MDN - [CSS variables - Browser support](#)

Elnevezési konvenciók

A változók elnevezésére érdemes időt szánni, mivel jó esetben egy hosszú kódban fogjuk használni fontos, hogy kifejező legyen. Mindent angolul, kisbetűvel írunk és a szavakat kötőjellel választjuk el.

MDN - [Using Custom Properties \(CSS variables\)](#)

4.21. Összetett szelektorok

Összetett - kombinált – szelektorok

Az összetett szelektorok használatával különböző elemek közötti kapcsolat és az elem a dokumentumban elfoglalt helye alapján tudunk formázni. A kombinált szelektorok nem helyettesítik az id-t vagy a classt, egy plusz dimenzióban tudjuk formázni a használatukkal a weboldalt.

Mikor használjuk?

A weboldal fejlesztésével egyre több egymásba ágyazott kódot írunk, és minél bonyolultabb és hosszabb a kód, annál nehezebb lesz elérni egyes elemeket.

Azért, hogy két hasonló elem közötti apró változtatáshoz ne kelljen új classt vagy id-t létrehozni, használjunk kombinált szelektorokat.

4.21.1. Univerzális szelektor

Bár nem tartozik az összetett szelektorok csoportjába, de mindenképp érdemes ebben a témakörben (is) szót ejteni róla.

```
* {
  padding: 100px; /*Az oldalon található összes elemre rákerül ez az érték */
}
```

A html/body szelektorok használata abban különbözik az univerzális szelektorétól, hogy utóbbi az oldal összes * eleméhez hozzáadja az értéket. Ha a body elemre tesszük rá a paddinget, akkor csak a body elem fogja megkapni ezt az értéket.

A body szelektor specifikusabb, így az felül fogja írni az univerzális szelektor stílusát.

*Az összes elembe nem tartoznak bele a ::before és ::after pszeudó szelektorok. Ha ezeket is a többi elemhez hasonlóan szeretnénk formázni, akkor adjuk hozzá explicit:

```
*::after {
  padding: 100px;
}
```


A * szelektor blokkonként is alkalmazható, bár ezt inkább csak akkor érdemes használni, ha vissza akarunk állítani blokkra ható értékeket.

```
<section class="first-section">
  <h3>This is first title</h3>
</section>
```

```
.first-section * {
  background-color: red;
}
```

4.21.2. Descendant Selector

Minden megjeleölt elemre érvényes, amelyet a kiválasztott elem/blokk tartalmaz.
Az alábbi példában a section elem minden p elemére érvényes lesz a formázás.

```
<section>
  <div>
    <p>First paragraph.</p>
  </div>
</section>
```

```
section p {
  padding: 10px;
}
```

4.21.3. Child selector

A leszármazott szerelktor elvén működik, több szelektort teszünk egymás után, de a relációs jellel meghatározzuk, hogy csak a közvetlen child elemét érje el.

Az alábbi kódban a csak a section elemek közvetlen p "gyerekei" kapják meg a formázást.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <section>
      <div>
        <p>First paragraph.</p>
        <p>Second paragraph</p>
        <section>
          <p>Not direct paragraph</p>
        </section>
      </div>
```

```
<p>First paragraph outside div.</p>
<p>Second paragraph outside div.</p>
</section>
</body>
</html>
```

```
section > p {
  background-color: seagreen;
}
```

First paragraph.

Second paragraph

Not direct paragraph

First paragraph outside div.

Second paragraph outside div.

4.21.4. Adjacent Sibling Selector

Ezzel a kombinálási módszerrel minden olyan elemet formázhatunk, amelyeknek közös a szülő eleme és szomszédos, egymás alatt lévő "testvér" elemek.

A két testvér elem szelektorát kell megadnunk, közöttük plusz jellel.

Az alábbi kombináció azokra a p elemekre lesz érvényes, melyek egy másik p elemet követnek.

```
<p>Ez egy bekezdés.</p>
<p>Ez egy másik bekezdés, de ez egyedi stílust kap.</p>
<div>Ez egy div, amely nem kap egyedi stílust.</div>
<p>Ez egy újabb bekezdés.</p>
```

```
p + p {
  color: red;
  font-weight: bold;
}
```

Ez egy bekezdés.

Ez egy másik bekezdés, de ez egyedi stílust kap.

Ez egy div, amely nem kap egyedi stílust.

Ez egy újabb bekezdés.

4.21.5. General Sibling Selector

Ez a kombináció kiválaszt minden olyan elemet, amely a megjelölt elemnek "testvére". Az első szelektor maga a megjelölt elem, tilde/hullámvonal jellel ~ választjuk el a testvér-elemtől.

```
<p>Ez egy bekezdés.</p>
<p>Ez egy másik bekezdés, és ez egyedi stílust kap.</p>
<div>Ez egy div.</div>
<p>Ez egy újabb bekezdés, és ez is egyedi stílust kap.</p>
<p>Ez egy újabb bekezdés, és ez is egyedi stílust kap.</p>
```

```
p ~ p {
  color: blue;
  font-style: italic;
}
```

Ez egy bekezdés.

Ez egy másik bekezdés, és ez egyedi stílust kap.

Ez egy div.

Ez egy újabb bekezdés, és ez is egyedi stílust kap.

Ez egy újabb bekezdés, és ez is egyedi stílust kap.

Opcionális olvasnivaló

[Better semantics with CSS Combinators and Selectors](#)

4.22. BEM

A BEM (Block, Element, Modifier) egy elnevezési konvenció a CSS-ben, amely segít jobban szervezni a stíluslapokat és megkönnyíti a komponensek újra felhasználhatóságát. Az alapötlete, hogy oszd fel a UI-t független blokkokra és elemekre.

Block: Egy független, újrafelhasználható funkcionális egység. Például egy gomb vagy fejléc.

```
.button {
  /* stílusok */
}
.header {
  /* stílusok */
}
```

Element: Egy blokkon belüli részegység. Az elemeket a blokk nevével és két alsóvonással jelöljük.

```
.header__title {
  /* a fejléc címének stílusai */
}
.header__nav {
```

```
/* a fejléc navigációjának stílusai */
}
```

Modifier: A blokkok vagy elemek állapotát, vagy viselkedését módosítja. A módosítókat a blokk vagy elem nevével és két kötőjellel jelöljük.

```
.button--large {
  /* nagy méretű gomb stílusai */
}
.header__nav--hidden {
  /* elrejtett navigáció stílusai */
}
```

A BEM módszer használata segít abban, hogy a kód strukturált és könnyen érthető legyen, valamint megkönnyíti a stílusok karbantartását és bővítését.

4.22.1. Összetett példa:

HTML kódban

```
<a class="btn btn--big btn--orange">
  <p class="btn__price">$9.99</p>
  <p class="btn__text">Subscribe</p>
</a>
```

CSS kódban

```
/* Block komponens*/
.product {
}
/* Az Elem, ami függ a blokk komponenstől */
.product __price {
  /* stílus */
}
/* Modifier, ami megváltoztatja az elem stílusát */
.product --orange {
  /* stílus */
}
.product --big {
  /* stílus */
}
```

Ne használj egynél több child class-t az elnevezésben.

Hivatalos weboldal - [BEM](#)